

DIGITAL LOGIC AND COMPUTER DESIGN

FINITE STATE MACHINES

Finite State Machine (FSM):

- A Finite State Machine (FSM) is a model used to design synchronous sequential circuits.
- It works by moving between a limited (finite) number of states based on the current inputs and the present state of the circuit.
- A circuit containing k registers can have 2^k different states because each register stores one bit of information. The FSM can be in only one state at a time.

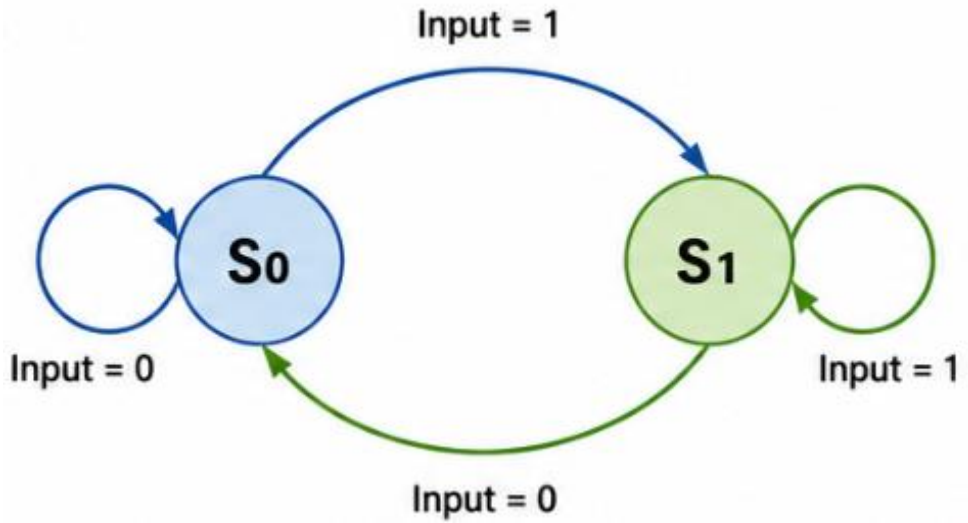
Number of Registers (k)	Possible States (2^k)
1	$2^1 = 2$
2	$2^2 = 4$
3	$2^3 = 8$
4	$2^4 = 16$

KEY POINT
Finite State Machine (FSM)
A digital circuit that stores its current state and changes to the next state on every clock edge. **"Finite"** means the machine has **only a limited number of possible states**.

FINITE STATE MACHINE

An FSM consists of:

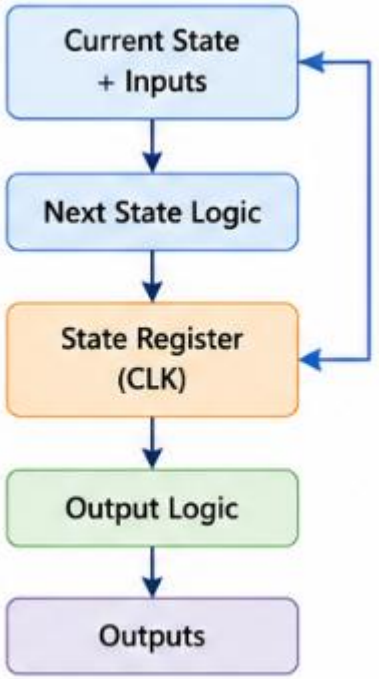
- **M Inputs** – Signals received from the external system.
- **N Outputs** – Signals produced by the circuit.
- **k-bit State Register** – Stores the current state of the FSM.
- **Clock Signal (CLK)** – Controls when the state changes.
- **Optional Reset Signal** – Returns the FSM to its initial state.



BASIC STRUCTURE OF FSM

A Finite State Machine consists of three main components that work together to determine the circuit's behavior.

1. Next State Logic
2. State Register (Memory)
3. Output Logic



1.Next State Logic:
Computes the next state of the FSM.

2. State Register (Memory)
Stores the present state of the FSM. Updates the stored state on every clock edge (CLK).

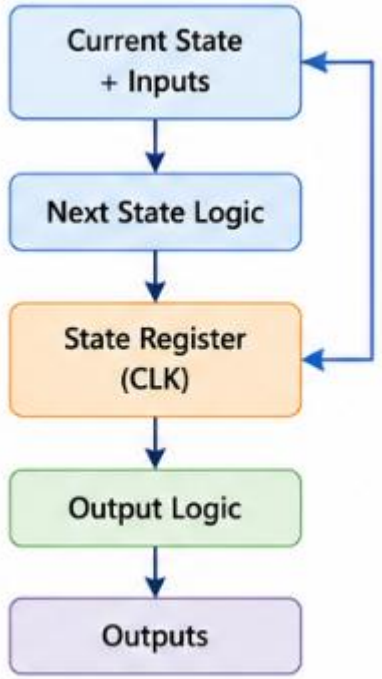
3. Output Logic
Generates the output signals of the FSM.

BASIC STRUCTURE OF FSM

The outputs are determined based on the type of FSM:

Moore Machine: Output depends only on the current state.

Mealy Machine: Output depends on the current state and current inputs.



Working Principle:

On every clock edge, the FSM:

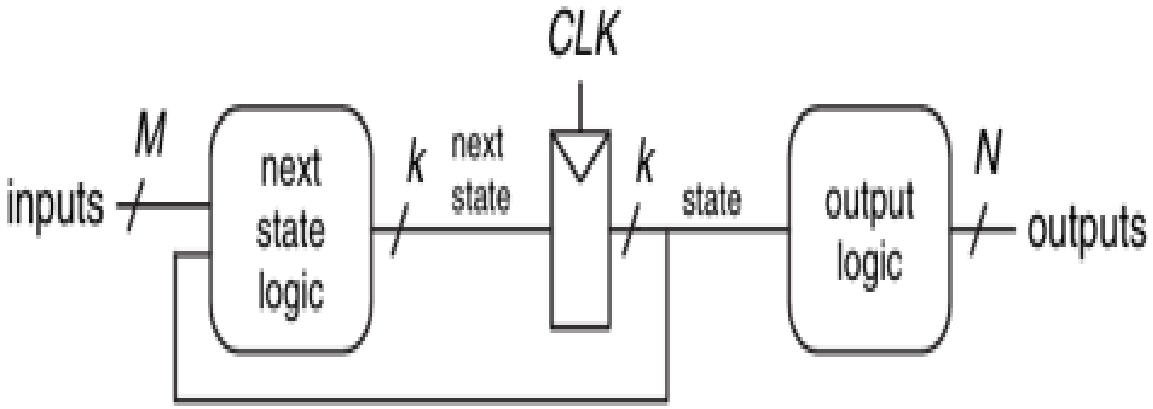
1. Reads the current state.
2. Accepts the current inputs.
3. Computes the next state.
4. Updates the state register.
5. Produces the required outputs.

TYPES OF FSM

Moore Machine:

Output depends only on the current state. Changes in input do not immediately affect the output. Output changes only after the state changes.

Output = f(Current State)



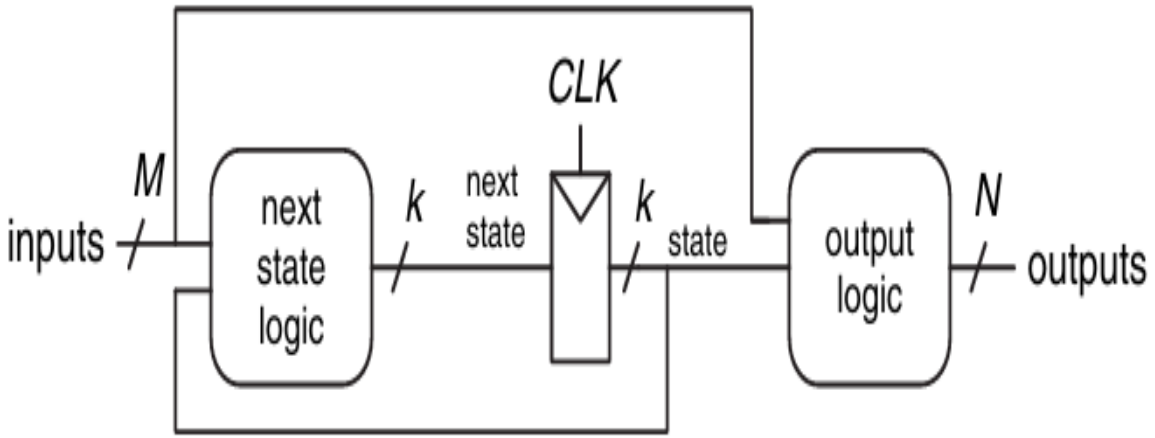
(a) Moore machine,

TYPES OF FSM

Mealy Machine

Output depends on both the current state and the current inputs. Output can change immediately when the input changes. Generally requires fewer states than a Moore machine.

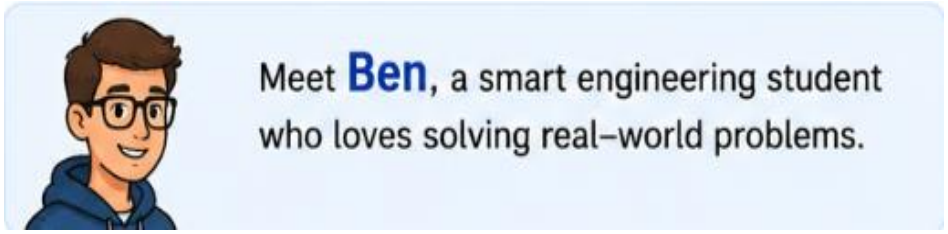
Output = f(Current State, Current Input)



(b) Mealy machine

DIGITAL LOGIC AND COMPUTER DESIGN

FSM DESIGN EXAMPLE



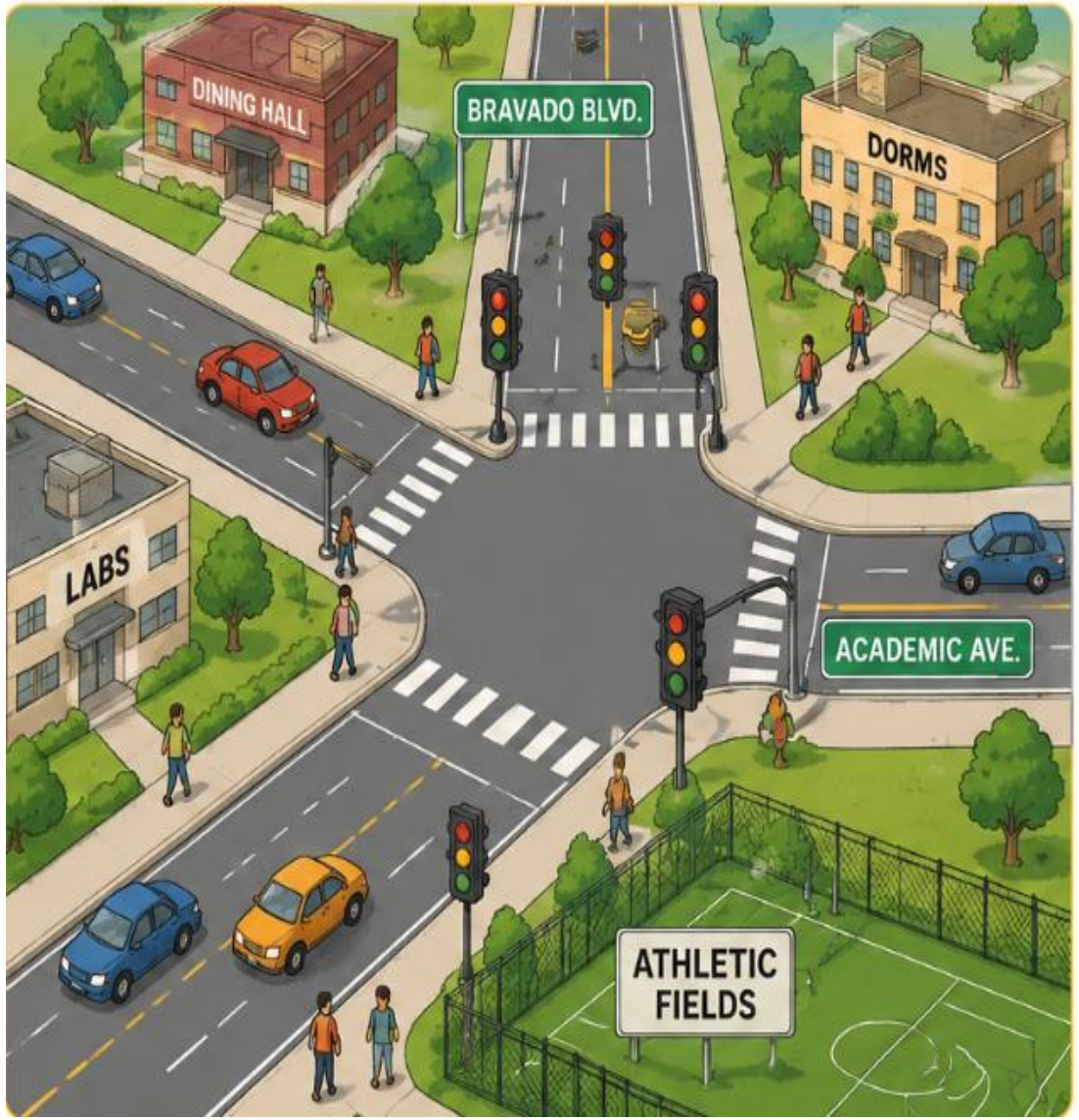
One day, while walking across campus, Ben notices the busy intersection between Academic Avenue and Bravado Boulevard.

Students are moving between the Dormitories and the Labs on Academic Ave.

Football players are traveling between the Athletic Fields and the Dining Hall on Bravado Blvd.

Since many people are busy and not paying attention, several accidents have already occurred at the intersection.

The Dean of Students asks Ben to design an automatic traffic light controller.

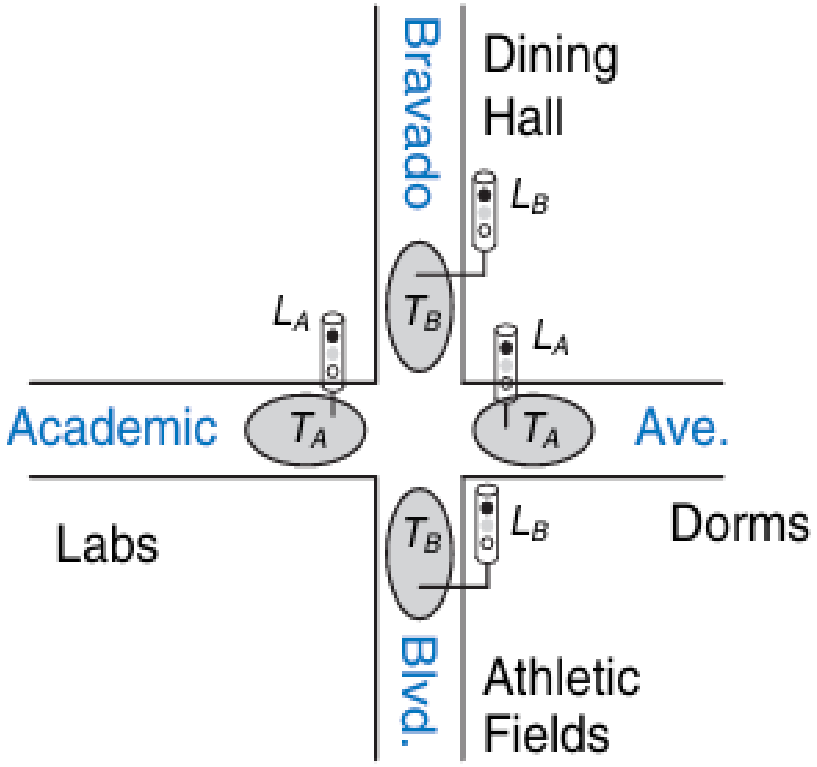


FSM DESIGN EXAMPLE

Design an FSM-based traffic light controller that uses traffic sensors to control the traffic lights and ensure safe, smooth traffic flow at the campus intersection.

Solution:

Ben decides to solve the problem using a Finite State Machine (FSM) that can monitor traffic and control the signals automatically.



Campus map

FSM DESIGN EXAMPLE

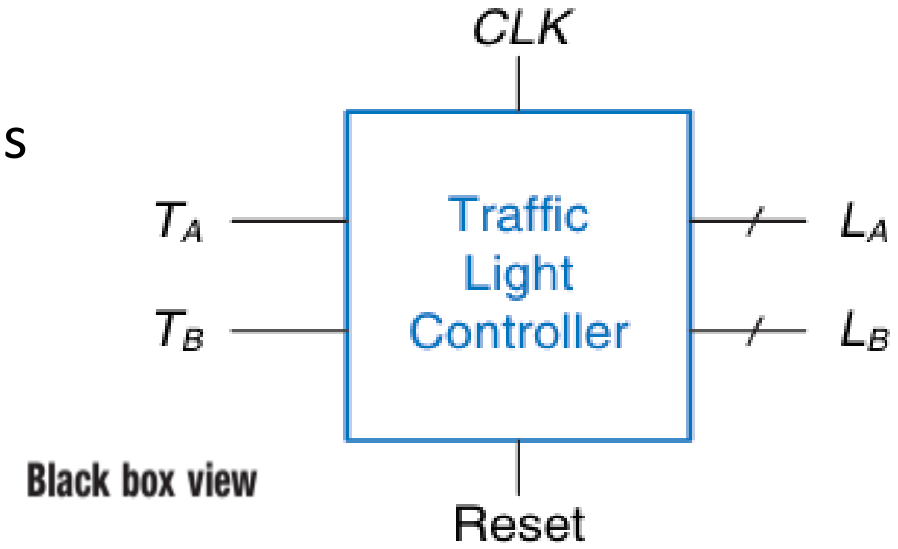
The FSM has:

- **2 Inputs:** T_A, T_B
- **2 Outputs:** LA, LB

Sensor	Location	Function
T_a	Academic Avenue	Detects vehicles on Academic Avenue.
T_β	Bravado Boulevard	Detects vehicles on Bravado Boulevard.

Traffic Light	Controls
L_a	Academic Avenue
L_β	Bravado Boulevard

- **Clock (CLK):** The controller checks traffic and updates the signals every 5 seconds.
- **Reset:** Initializes the FSM to its starting state when the controller is turned on.



Black box view

STATE TRANSITION DIAGRAM

Overview :

Each circle represents a state, and the directed arrows represent the transitions between states based on the traffic sensor inputs.

STATES:

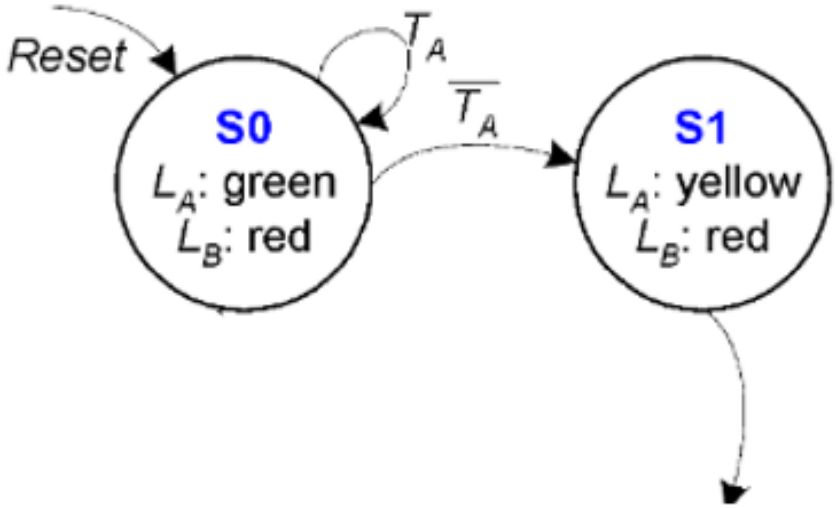
- **Moore FSM:** outputs labeled in each state
- **States:** Circles
- **Transitions:** Arcs

State	Academic Ave. (L_A)	Bravado Blvd. (L_B)
S0	green	red
S1	yellow	red
S2	red	green
S3	red	yellow

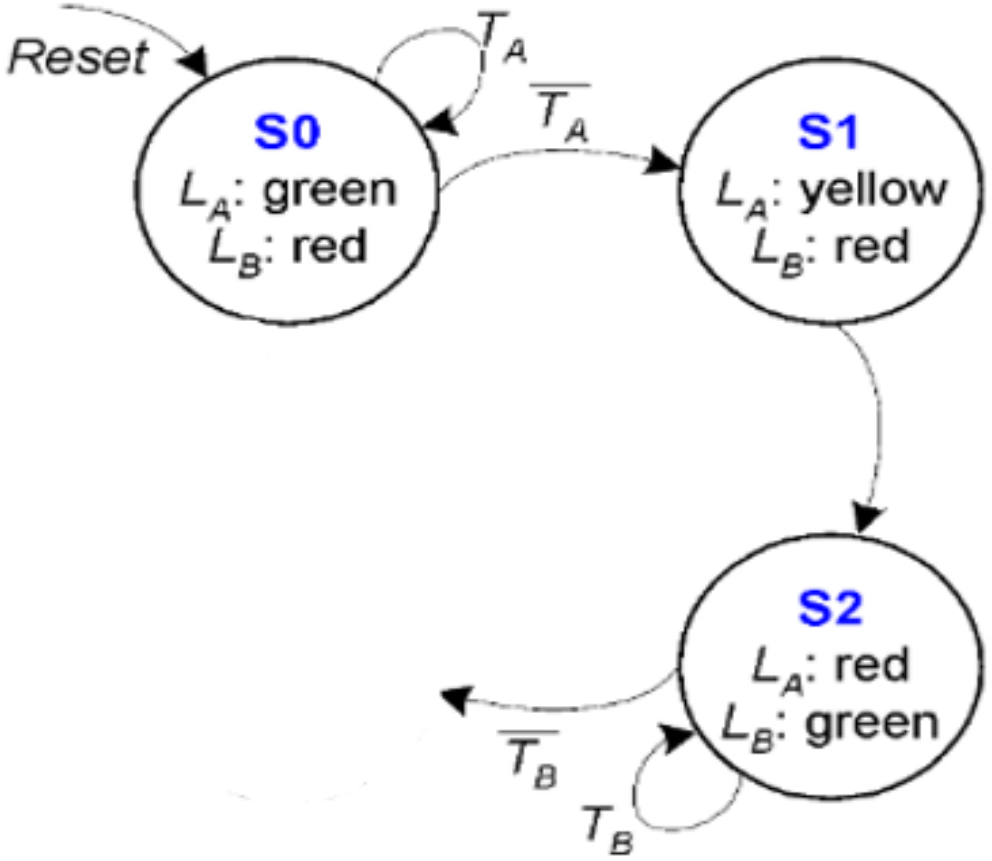
STATE TRANSITION DIAGRAM



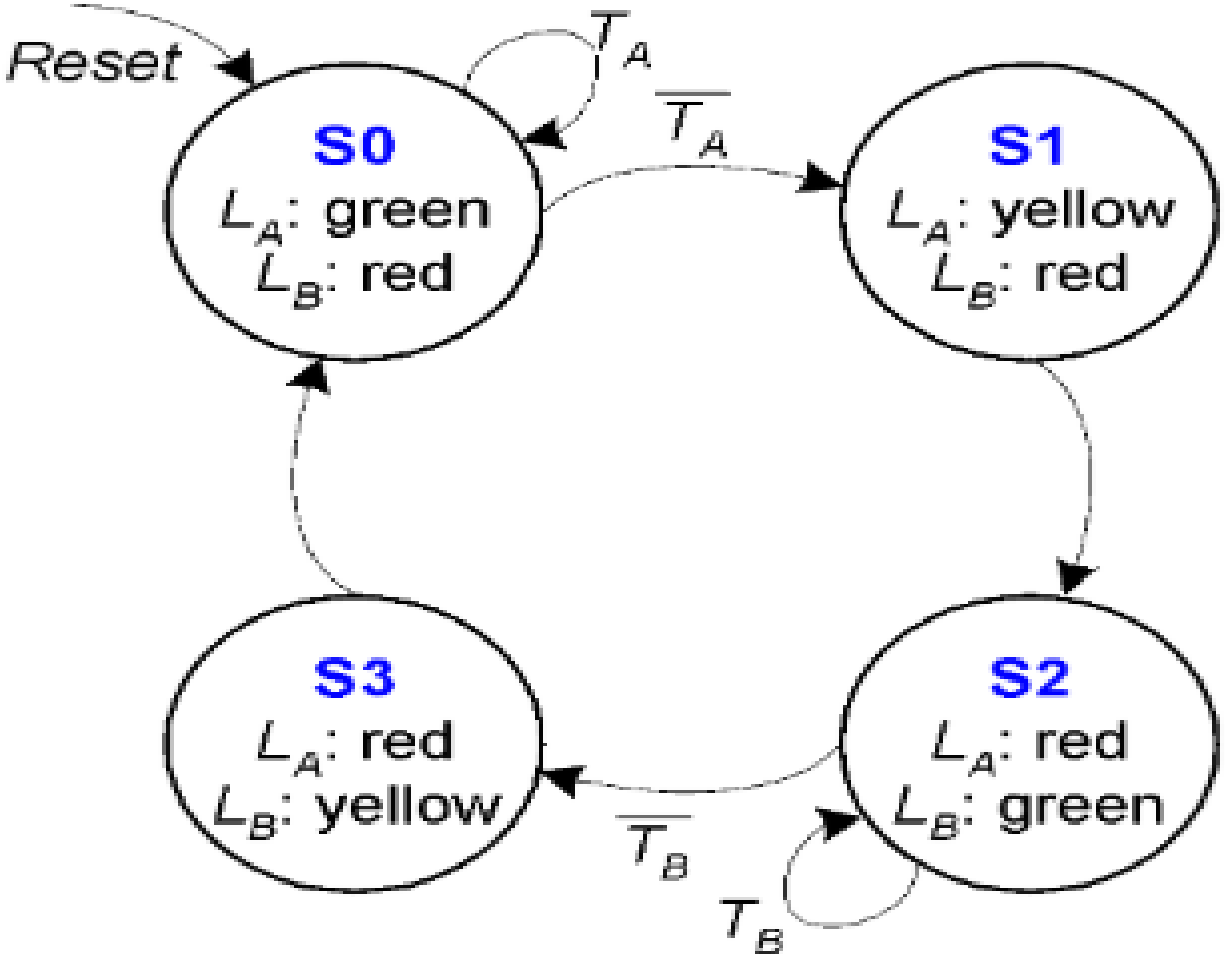
STATE TRANSITION DIAGRAM



STATE TRANSITION DIAGRAM



STATE TRANSITION DIAGRAM



STATE TRANSITION DIAGRAM

PES UNIVERSITY

State Transitions

Reset→S0

The FSM always enters State S0 whenever the Reset signal is activated.

S0 → S1: If $T_A = 0$, the controller changes from S0 to S1.

S1→S2: After 5 seconds, the controller automatically changes from S1 to S2.

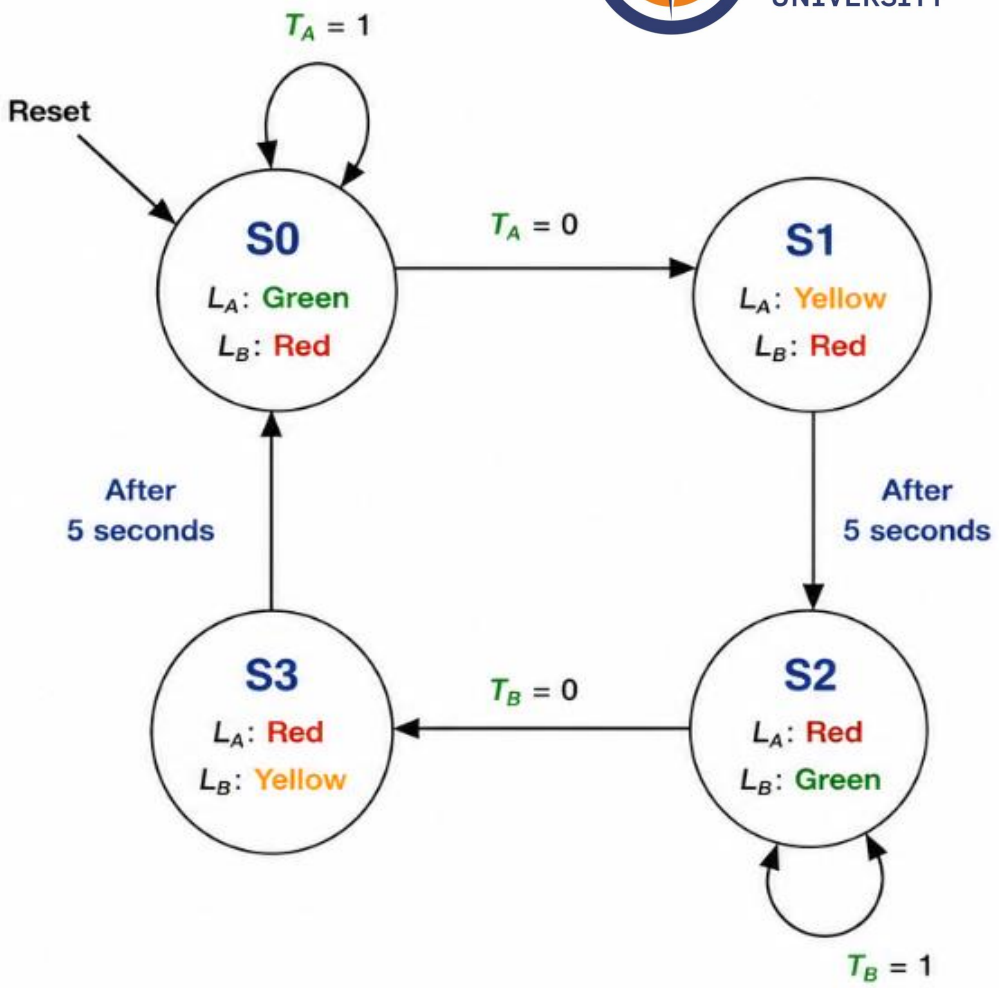
S2 → S3: If $T_B = 0$, the controller changes from S2 to S3.

S3 → S0: After 5 seconds, the controller returns to S0, completing one traffic signal cycle.

Conditions for Remaining in the Same State:

S0 → S0 when $T_A = 1$

S2 → S2 when $T_B = 1$



State transition diagram

STATE TRANSITION TABLE



After creating the State Transition Diagram, Ben converts it into a State Transition Table. The table specifies the next state of the FSM for every combination of the current state and input conditions.

The State Transition Table is a bridge between the graphical FSM representation and the digital hardware implementation.

X (Don't Care) :means that the value of that input does not affect the next state.

Examples:

- In S1, the FSM always moves to S2, regardless of T_A or T_B.
- In S3, the FSM always returns to S0, regardless of the sensor inputs.

Table 3.1 State transition table

Current State S	Inputs T_A	Inputs T_B	Next State S'
S0	0	X	S1
S0	1	X	S0
S1	X	X	S2
S2	X	0	S3
S2	X	1	S2
S3	X	X	S0

STATE AND OUTPUT ENCODING

Before implementing the FSM using digital hardware, the states and outputs must be represented in binary form. This process is called Encoding.

1. State Encoding

Each state is assigned a unique binary code.

Table 3.2 State encoding

State	Encoding $S_{1:0}$
S0	00
S1	01
S2	10
S3	11

2. Output Encoding

Each traffic light color is also represented using binary values.

Table 3.3 Output encoding

Output	Encoding $L_{1:0}$
green	00
yellow	01
red	10

STATE AND OUTPUT ENCODING

Why Encoding?

- Converts symbolic states into binary values.
- Makes the FSM compatible with digital circuits.
- Simplifies the derivation of Boolean equations.

Key Observation

- Every state and output now has a unique binary representation
- Encoding transforms the FSM from a conceptual model into a form that can be implemented using digital hardware.

STATE AND OUTPUT ENCODING

After assigning binary codes to each state, the State Transition Table is updated using these binary encodings.

Table 3.4 State transition table with binary encodings

Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S_1'	S_0'
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

Table 3.5 Output table

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

NEXT STATE LOGIC

The revised table defines the next state (S_1', S_0') as a function of the Current State (S_1, S_0) *and* Inputs (T_A, T_B)

$$S_1' = \overline{S}_1 S_0 + S_1 \overline{S}_0 \overline{T}_B + S_1 \overline{S}_0 T_B$$

$$S_0' = \overline{S}_1 \overline{S}_0 \overline{T}_A + S_1 \overline{S}_0 \overline{T}_B$$

- The Next-State Equations determine which state the FSM should move to based on the current state and the sensor inputs.
- The Boolean equations obtained from the state transition table can be simplified using Karnaugh Maps (K-Maps) or by inspection.
- This reduces the number of logic gates, makes the circuit simpler and improves hardware efficiency.

$$S_1' = S_1 \oplus S_0$$

$$S_0' = \overline{S}_1 \overline{S}_0 \overline{T}_A + S_1 \overline{S}_0 \overline{T}_B$$

OUTPUT LOGIC

Similarly the output table using the binary encoding is Table 3.5

Output Equations

$$L_{A1} = S_1$$
$$L_{A0} = \bar{S_1} S_0$$
$$L_{B1} = \bar{S_1}$$
$$L_{B0} = S_1 S_0$$

Table 3.5 Output table

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

COMPLETE FSM LOGIC CIRCUIT

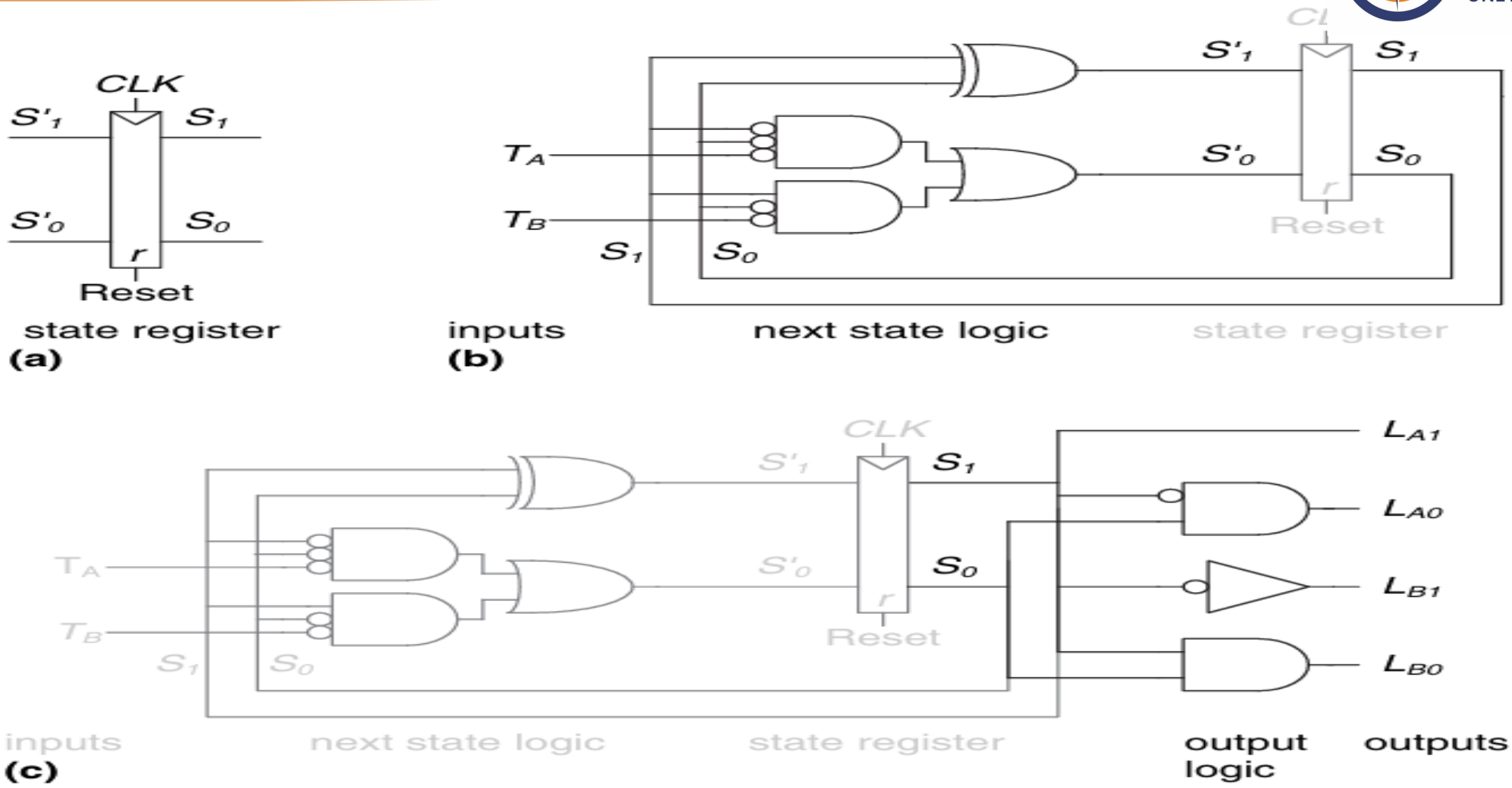


Figure 3.26 State machine circuit for traffic light controller

STATE ENCODING

- State Encoding is the process of assigning binary codes to the states of a Finite State Machine (FSM).

- The choice of encoding directly affects the hardware implementation, including the number of logic gates, propagation delay, and overall circuit complexity.

Different state encodings can produce different:

- circuit implementations
- numbers of logic gates
- propagation delays
- hardware complexity

Therefore, selecting an appropriate encoding helps in designing an efficient FSM.

STATE ENCODING

- Finding the optimal encoding is difficult because it requires checking all possible encoding combinations, which becomes impractical as the number of states increases.
- Instead, designers usually choose a suitable encoding based on the application or use Computer-Aided Design (CAD) tools to help select an efficient encoding.
- The choice of state encoding influences the speed, size, and complexity of the final digital circuit.

BINARY ENCODING

In Binary Encoding, each state of the FSM is represented using a binary number.
For an FSM with K states, the number of bits required is:

$$\boxed{log_2 K}$$

Only the minimum number of bits needed to represent all states is used.
Ex:For 4 states

State	Binary Code
S0	00
S1	01
S2	10
S3	11

Advantages

- Uses fewer state bits and requires fewer flip-flops
- Efficient hardware utilization

ONE HOT ENCODING

In One-Hot Encoding, each state is assigned a separate bit.
Only one bit is HIGH (1) at any given time, while all other bits remain LOW (0).
For K states, K bits are required.

Example: For 3 states

State	Encoding
S0	001
S1	010
S2	100

Advantages

Simpler combinational logic and faster implementation

Disadvantages

Requires more flip-flops and higher hardware cost

DESIGNING A DIVIDE-BY-3-COUNTER

To understand how different state encoding techniques are applied, consider the design of a Divide-by-3 Counter.

A Divide-by-3 Counter is a sequential circuit that reduces the frequency of the input clock by a factor of 3.

System Description:

- The FSM has one output (Y) and three states S0, S1, and S2 .
- The output Y becomes HIGH (1) for one clock cycle out of every three clock cycles.

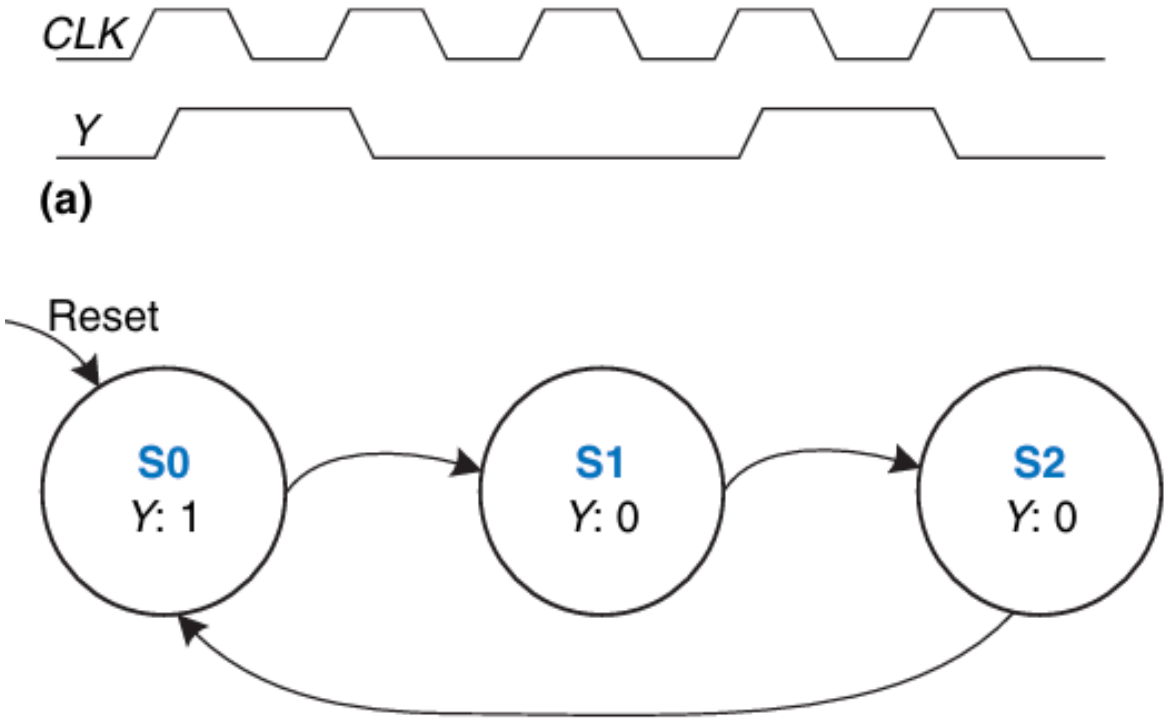


Figure 3.28 Divide-by-3 counter (a) waveform and (b) state transition diagram

DESIGNING A DIVIDE-BY-3-COUNTER

Operation:

The counter continuously cycles through the following sequence:

S0 → S1 → S2 → S0 → . . .

This repeating sequence divides the clock frequency by 3.

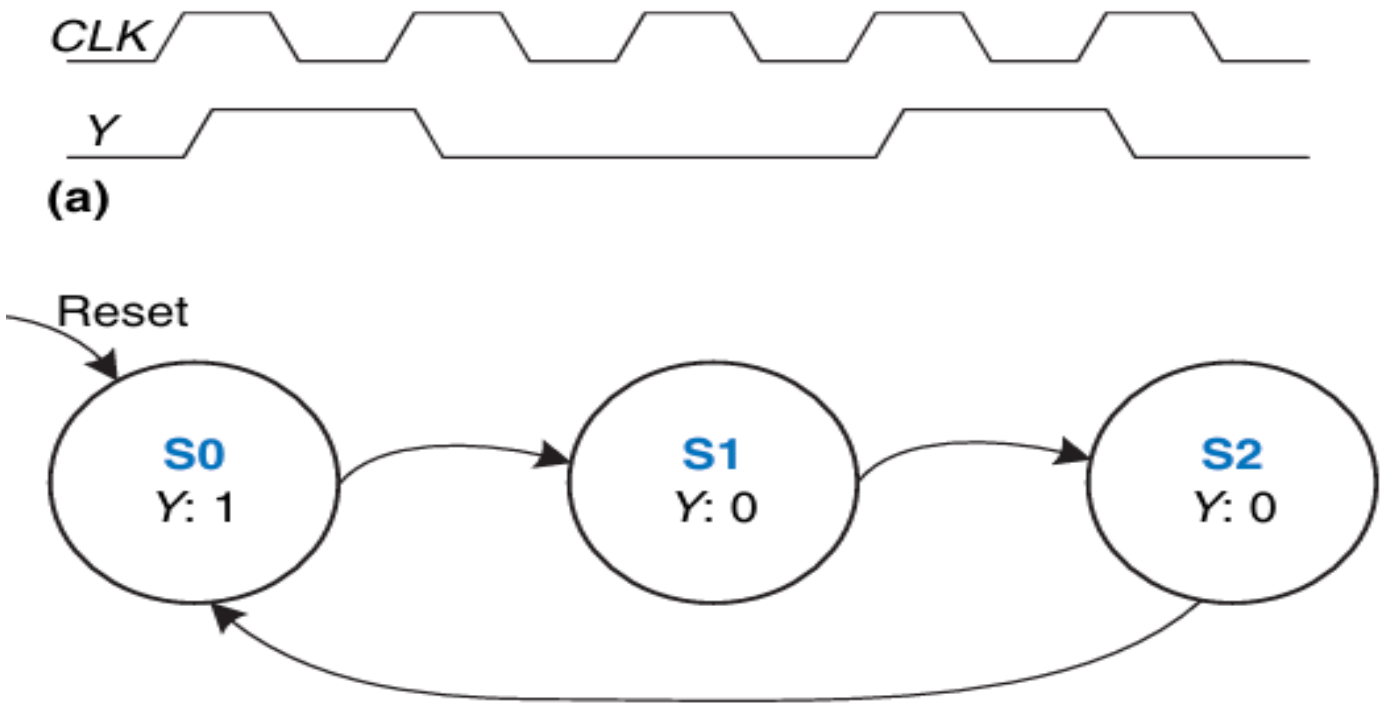


Figure 3.28 Divide-by-3 counter (a) waveform and (b) state transition diagram

STATE TRANSITION TABLE

1. State Transition Table:

The State Transition Table defines the next state for every current state.

Table 3.6 Divide-by-3 counter state transition table

Current State	Next State
S0	S1
S1	S2
S2	S0

OUTPUT TABLE

2. Output Table:

The Output Table specifies the output value associated with each state.

Table 3.7 Divide-by-3 counter output table

Current State	Output
S0	1
S1	0
S2	0

BINARY VS ONE HOT ENCODING

To implement the Divide-by-3 Counter in hardware, each state must be assigned a binary code. Two commonly used encoding techniques are Binary Encoding and One-Hot Encoding.

Table 3.8 One-hot and binary encodings for divide-by-3 counter

State	One-Hot Encoding			Binary Encoding	
	S_2	S_1	S_0	S_1	S_0
S0	0	0	1	0	0
S1	0	1	0	0	1
S2	1	0	0	1	0

STATE TRANSITION TABLE USING BINARY ENCODING

Binary State Transition Table

- The Binary State Transition Table uses the 2-bit binary codes assigned to each state.
- States are represented using 2 bits.

Table 3.9 State transition table with binary encoding

Current State		Next State	
S_1	S_0	S_1'	S_0'
0	0	0	1
0	1	1	0
1	0	0	0

STATE TRANSITION TABLE USING ONE-HOT ENCODING

PES UNIVERSITY

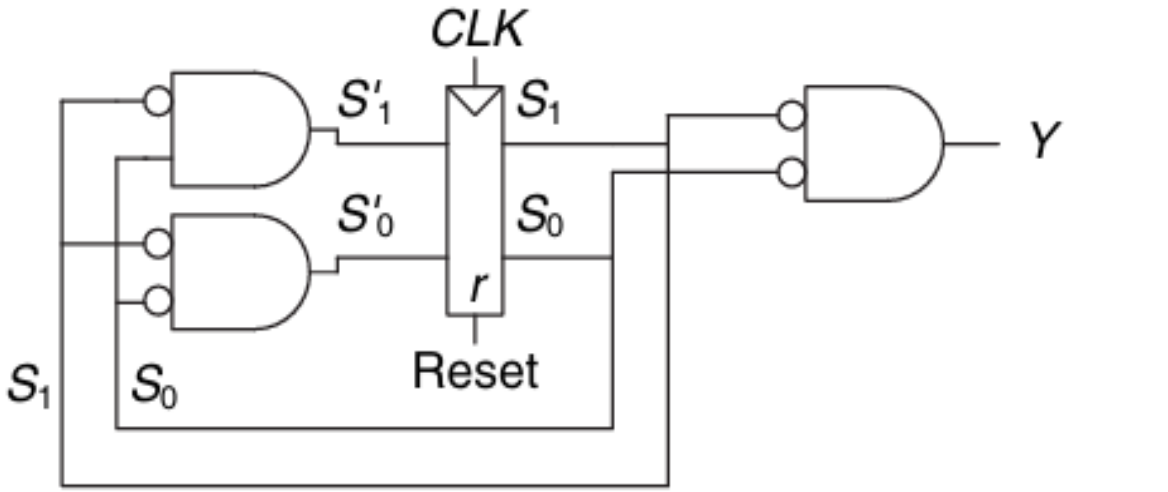
One-Hot State Transition Table

- The One-Hot State Transition Table uses one flip-flop for each state
- Each state is represented by a unique active bit. Only one bit is HIGH at any time.

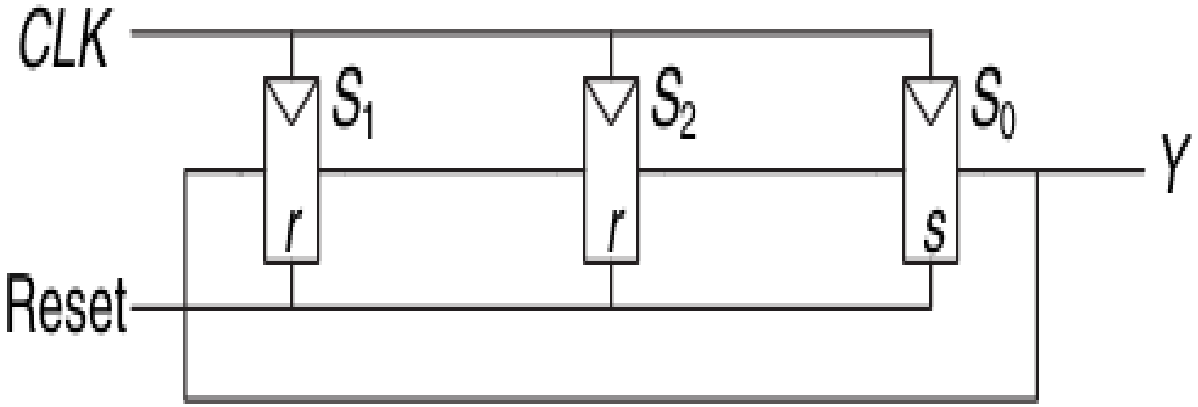
Table 3.10 State transition table with one-hot encoding

Current State			Next State		
S_2	S_1	S_0	S'_2	S'_1	S'_0
0	0	1	0	1	0
0	1	0	1	0	0
1	0	0	0	0	1

Binary Encoding Implementation:



One-Hot Encoding Implementation:





DIGITAL LOGIC AND COMPUTER DESIGN

FINITE STATE MACHINES

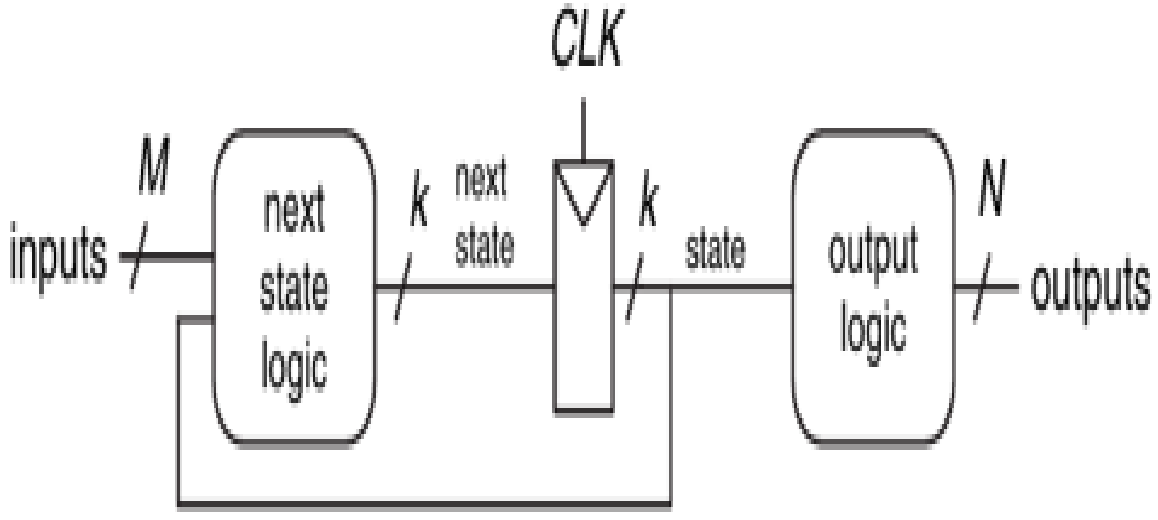
TYPES OF FSM

Moore Machine:

Output depends only on the current state. Changes in input do not immediately affect the output. Output changes only after the state changes. Outputs are written inside the state circles.

Formula

Output = f(Current State)



2 Finite state
(a) Moore machine,

KEY POINT

Moore machines usually require more states than Mealy machines for the same problem.

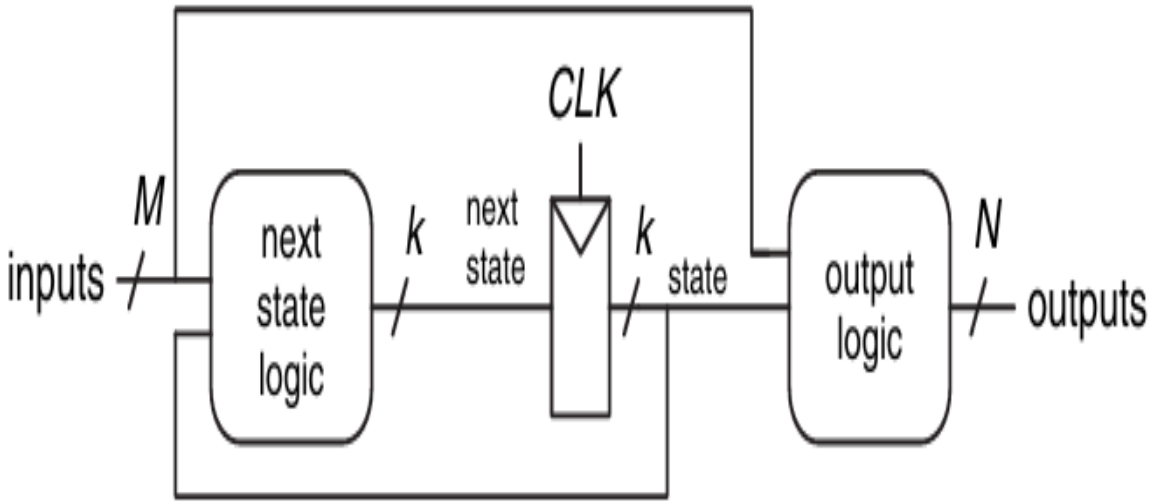
TYPES OF FSM

Mealy Machine

Output depends on both the current state and the current inputs. Output can change immediately when the input changes. Generally requires fewer states than a Moore machine. Outputs are written on the transition arrows.

Formula

Output = f(Current State, Current Input)



(b) Mealy machine

MOORE VS MEALY MACHINES

Scenario:

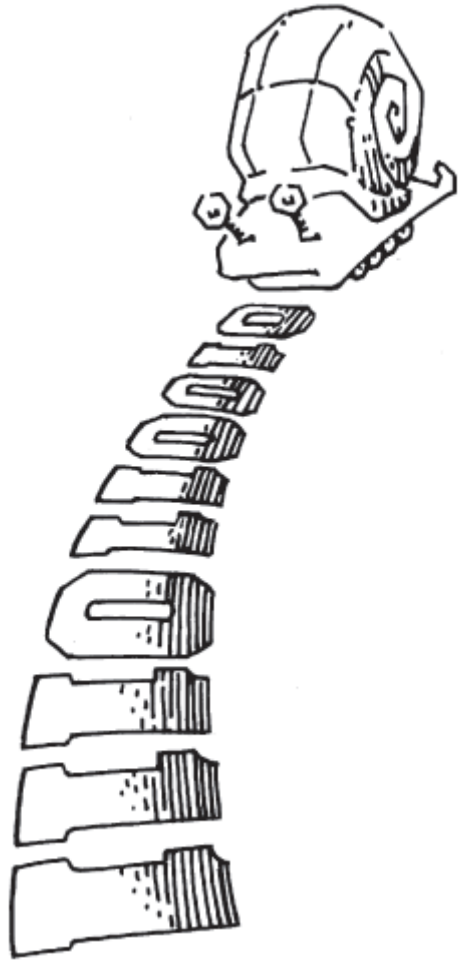
Alyssa P. Hacker owns a pet robotic snail controlled by a Finite State Machine (FSM).The snail moves over a paper tape containing a sequence of 1s and 0s.

At every clock cycle:

The snail moves one position forward.

It remembers the last two bits it has read.

The snail smiles when a specific bit pattern is detected.



Input: [A] Current bit read by the snail.

Output:[Y]

- **Y = 1** → Snail smiles.
- **Y = 0** → Snail does not smile.

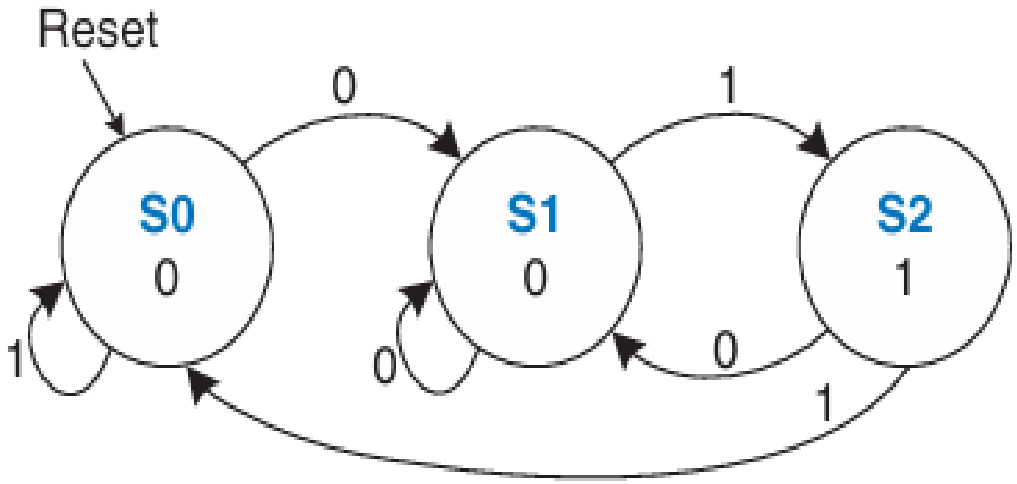
MOORE VS MEALY MACHINES

Objective

Design both Moore Machine and Mealy Machine .Compare both implementations.

Moore FSM Design :

The Moore Machine requires three states. Each state represents the previous input history.The output depends only on the current state.

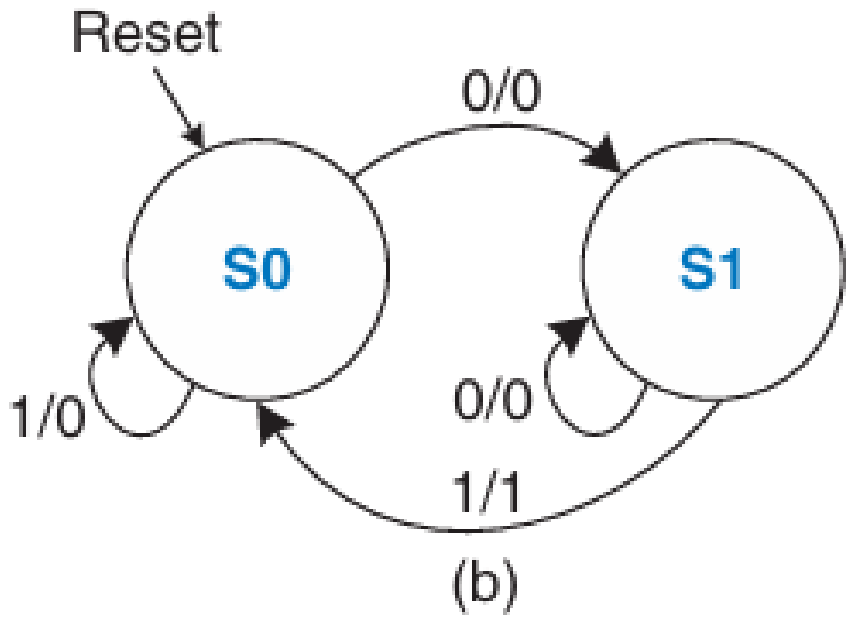


FSM state transition diagrams: (a) Moore machine,

Mealy Machine Design:

The Mealy Machine requires only two states. Outputs are generated during state transitions.

Each transition is labelled as **Input / Output**



MOORE TRANSITION AND OUTPUT TABLE

Table 3.11 Moore state transition table

Current State S	Input A	Next State S'
S0	0	S1
S0	1	S0
S1	0	S1
S1	1	S2
S2	0	S1
S2	1	S0

Table 3.12 Moore output table

Current State S	Output Y
S0	0
S1	0
S2	1

MOORE TRANSITION AND OUTPUT TABLE WITH BINARY ENCODING

To implement the Moore Machine in digital hardware,each state is assigned a binary code.Using these binary encodings,the transition table and output table are rewritten.

Table 3.13 Moore state transition table with state encodings

Current State S_1	Current State S_0	Input A	Next State S'_1	Next State S'_0
0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0

Table 3.14 Moore output table with state encodings

Current State S_1	Current State S_0	Output Y
0	0	0
0	1	0
1	0	1

MOORE FSM

Table 3.13 Moore state transition table with state encodings

Current State S_1	Current State S_0	Input A	Next State S'_1	Next State S'_0
0	0	0	0	1
0	0	1	0	0
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0

Table 3.14 Moore output table with state encodings

Current State S_1	Current State S_0	Output Y
0	0	0
0	1	0
1	0	1

Equations:

$$S'_1 = S_0 A$$
$$S'_0 = \overline{A}$$

Equation 3.3.1

$$Y = S_1$$

MEALY TRANSITION AND OUTPUT TABLE

The Mealy Machine requires only **one state bit**.

Table 3.15 Mealy state transition and output table

Current State S	Input A	Next State S'	Output Y
S0	0	S1	0
S0	1	S0	0
S1	0	S1	0
S1	1	S0	1

Table 3.16 Mealy state transition and output table with state encodings

Current State S_0	Input A	Next State S'_0	Output Y
0	0	1	0
0	1	0	0
1	0	1	0
1	1	0	1

MOORE VS MEALY MACHINES

Comparison:

Moore Machine	**Mealy Machine**
More states	Fewer states
Stable output	Faster response
Output depends only on state	Output depends on state and input
One clock delay	Immediate output

- Choose a Moore Machine when stable outputs are important.
- Choose a Mealy Machine when faster output response is required.



DIGITAL LOGIC AND COMPUTER DESIGN

FINITE STATE MACHINES

FACTORING STATE MACHINES

As the complexity of a Finite State Machine (FSM) increases, designing and maintaining it becomes difficult.
Instead of designing one large FSM, the system can be divided into multiple smaller FSMs that work together.
This design approach is called Factoring of State Machines.

How Does It Work?

- A complex FSM is split into smaller interacting FSMs.
- The output of one FSM can be used as the input to another FSM.
- This creates a hierarchical and modular design.

FACTORED AND UNFACTORED STATE MACHINES

To understand the concept of factoring, consider the traffic light controller introduced earlier.

The controller is now modified to include a Parade Mode, which keeps the Bravado Boulevard traffic light green while a parade is in progress.

Additional Inputs:

Two new control inputs are introduced:

- **P (Parade)** – Enables Parade Mode.
- **R (Resume)** – Exits Parade Mode.



(a) single traffic light controller FSM

FACTORED AND UNFACTORED STATE MACHINES

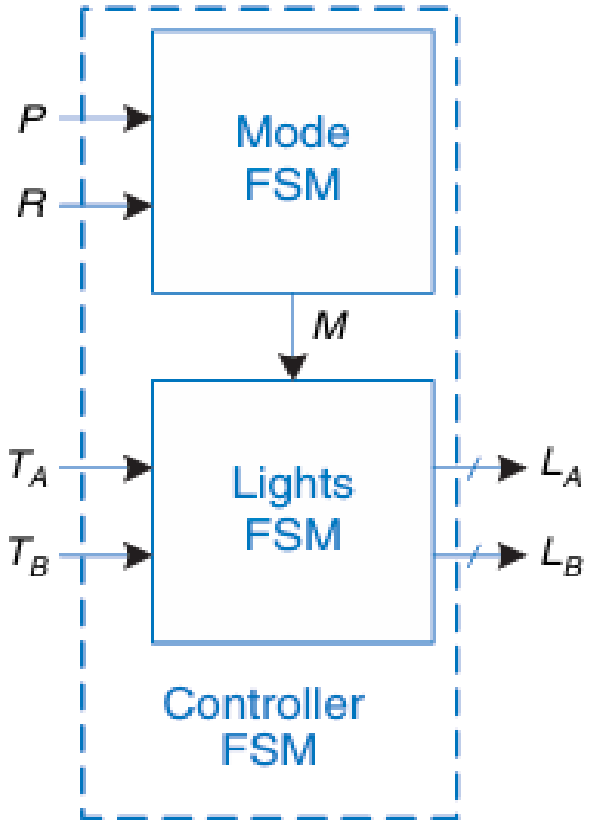
Controller Behaviour:

- When P = 1, the controller enters Parade Mode.
- During Parade Mode, the controller follows its normal sequence until LB becomes green.
- Once LB is green, it remains green throughout the parade.
- When R = 1, the controller exits Parade Mode and resumes normal traffic operation.

Objective:

Design the controller using:

- A single FSM (Unfactored).
- Multiple interacting FSMs (Factored).



(b) factored designs for modified traffic light controller FSM

UNFACTORED FSM

Single FSM Approach:

In the unfactored design, a single FSM controls both:

- Normal traffic operation.
- Parade mode.

State Organization

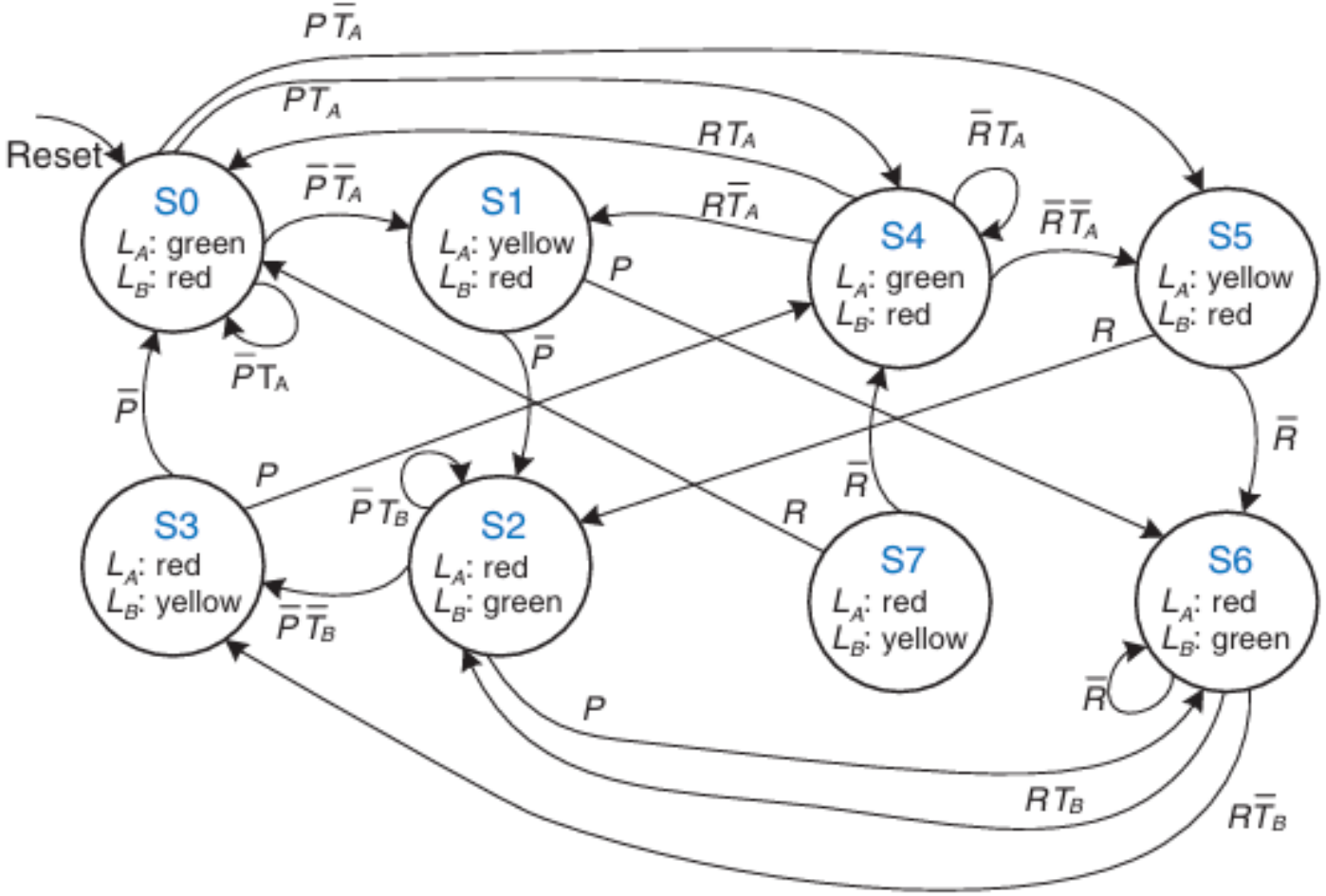
- S0 – S3 → Normal Mode.
- S4 – S7 → Parade Mode.

The two halves of the state diagram are almost identical, except that during Parade Mode the controller remains in State S6 while LB remains green.

Characteristics

- One large FSM.
- More states.
- Complex transitions.
- Difficult to modify and debug.

UNFACTORED FSM



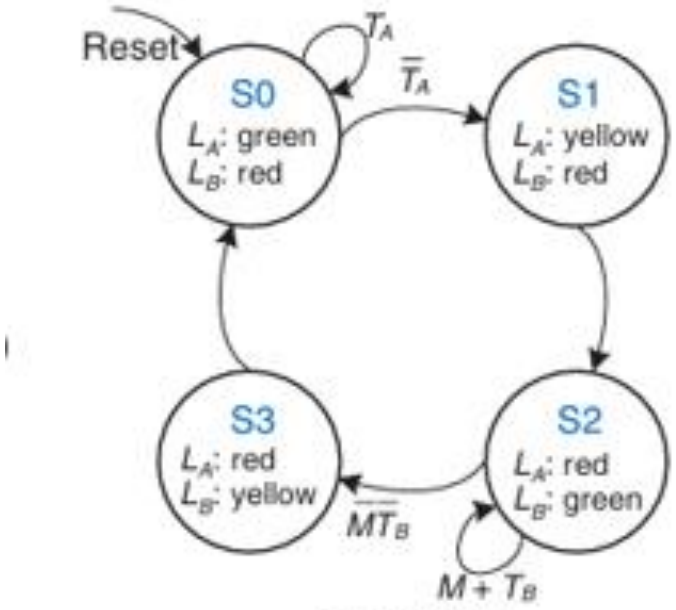
(a) unfactored,

Breaking the FSM into Modules:
Instead of using one large FSM, the controller is divided into two smaller FSMs.

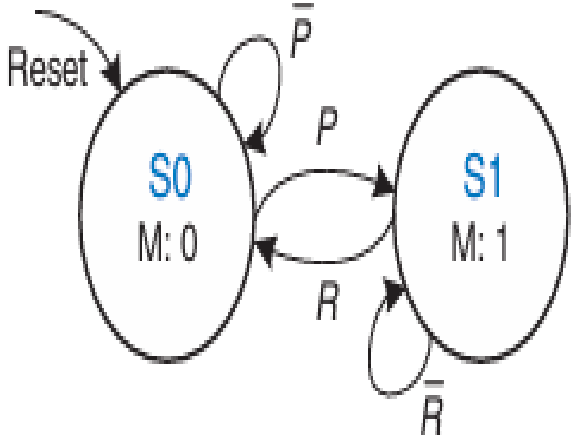
Mode FSM:
Tracks whether the controller is operating in:
- Normal Mode
- Parade Mode

Produces output: M

Lights FSM
Controls the traffic lights using:
- Traffic Sensors (TA, TB)
- Mode Signal (M)



Lights FSM



Mode FSM

(b) factored

FACTORED VS UNFACTORED FSM

Unfactored FSM	**Factored FSM**
Single large FSM	Multiple smaller FSMs
More states	Fewer states per FSM
Complex transitions	Simpler transitions
Difficult to debug	Easier to debug
Less modular	Highly modular
Harder to modify	Easy to extend

DERIVING AN FSM FROM SCHEMATIC

PES UNIVERSITY

STEPS FOR DERIVING FSM:

1.Examine the circuit and identify Inputs,Outputs ,State bits.

2.Write the next-state and output equations.

3.Create the next-state and output tables.

4.Remove unreachable states.

5.Assign symbolic names to valid states.

6.Draw the state transition diagram.

7.Describe the overall function of the FSM.

EXAMPLE

Scenario:

Alyssa P. Hacker discovers that the keypad lock in her house has been rewired, and the original unlock code no longer works.She finds a circuit diagram attached to the keypad and suspects it is controlled by a Finite State Machine (FSM).To determine the correct unlock sequence, she decides to derive the FSM from the circuit.

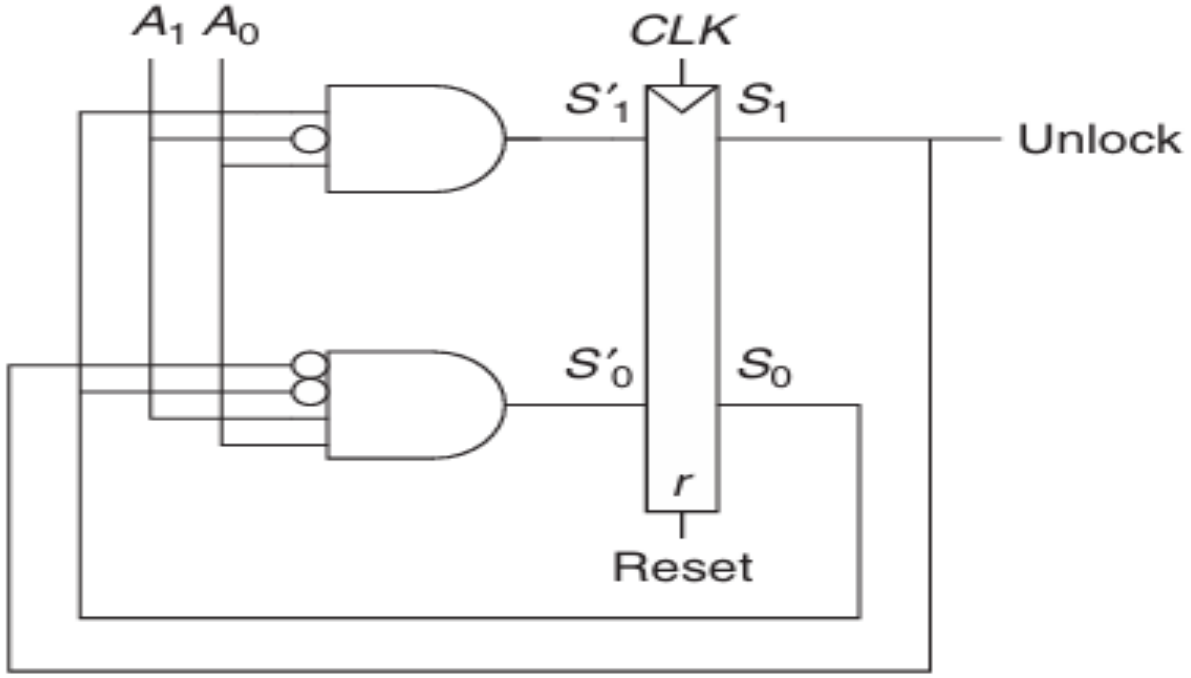


Figure 3.35 Circuit of found FSM for Example 3.9

STEP1: IDENTIFICATION

Objective

Determine the FSM states ,State transitions ,Output behaviour and Overall function of the circuit

Step 1 – Identify Inputs, Outputs & State Equations:

From the circuit:

Inputs: A_1 ,A_0

Output: Unlock

State Bits: S_1 ,S_0

Next-State Equations:

$$S_1' = S_0A_1A_0$$

$$S_0' = \overline{S_1}S_0A_1A_0 \qquad \text{Equation (3.12)}$$

$$\text{Unlock} = S_1$$

STEP2-CREATE TABLES

Step 2 – Create State Tables:

Table 3.18 Output table derived from circuit in Figure 3.35

Current State S_1	S_0	Output *Unlock*
0	0	0
0	1	0
1	0	1
1	1	1

Table 3.17 Next state table derived from circuit

Current State S_1	S_0	Input A_1	A_0	Next State S_1'	S_0'
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	0	0
1	1	0	0	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	1	0	0

STEP3-REDUCTION

Step 3 – Reduce the State Tables:

Some state combinations are never reached during operation.These are called Unreachable States.Removing these states simplifies the FSM.

Table 3.19 Reduced next state table

Current State S_1	Current State S_0	Input A_1	Input A_0	Next State S_1'	Next State S_0'
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	1
0	1	0	0	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	X	X	0	0

Table 3.20 Reduced output table

Current State S_1	Current State S_0	Output *Unlock*
0	0	0
0	1	0
1	0	1

STEP4-STATE ASSIGNMENT

Step 4 – Assign Symbolic State Names:

After removing unreachable states, each valid state is assigned a symbolic name.

Binary State	Symbolic Name
00	S0
01	S1
10	S2

Step 4 – Assign Symbolic State Names:

Table 3.21 Symbolic next state table

Current State S	Input A	Next State S'
S0	0	S0
S0	1	S0
S0	2	S0
S0	3	S1
S1	0	S0
S1	1	S2
S1	2	S0
S1	3	S0
S2	X	S0

Table 3.22 Symbolic output table

Current State S	Output *Unlock*
S0	0
S1	0
S2	1

STEP5-STATE TRANSITION TABLE

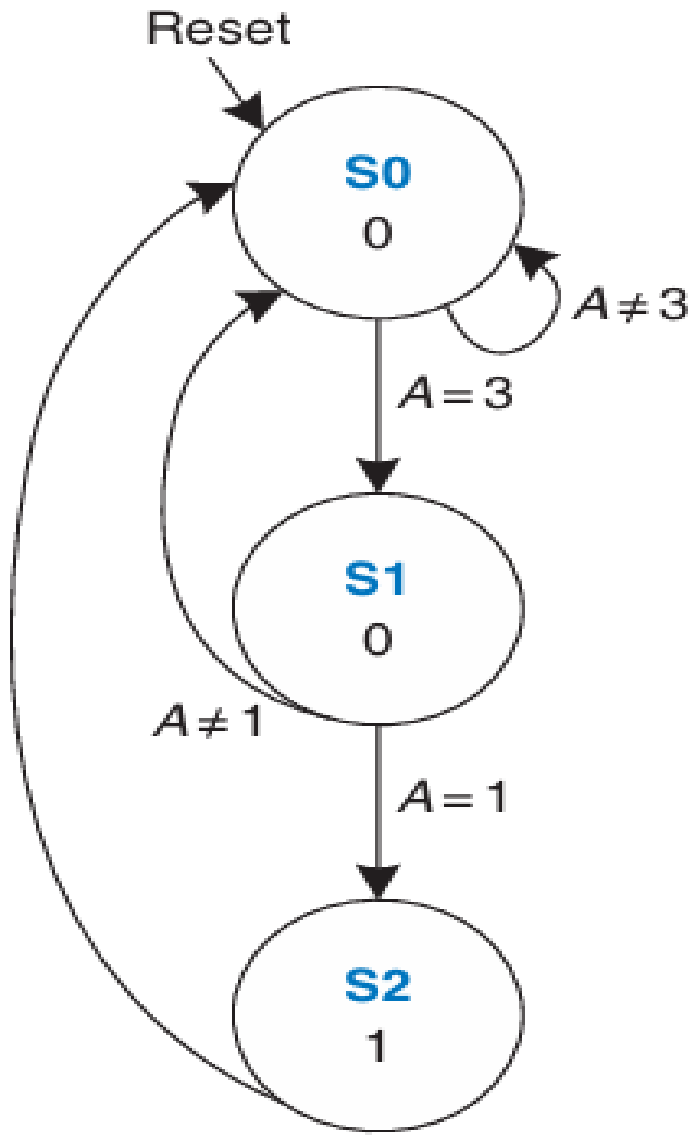
Step 5 – Draw the State Transition Diagram:

Using the symbolic tables, the final State Tra Diagram is constructed.

Overall Behaviour:

The FSM unlocks the door only after detecting the correct input sequence.

After unlocking, it returns to the initial state, ready for the next input sequence



PES UNIVERSITY

FSM Design Procedure

General Steps

1.Identify the inputs and outputs.

1.Draw the State Transition Diagram.

1.Create:
- State Transition Table
- Output Table

4.Select a suitable State Encoding.

4.Write:
- Next-State Equations
- Output Equations

6.Design the circuit schematic.

QUIZ

In **Example** the output **Unlock** is given by

$$Unlock = S_1$$

This indicates that the output depends on

A) Current input only

B) Current state only

C) Current input and current state

D) Next state only

Answer:B

2.An FSM is implemented using 5 flip-flops with Binary Encoding.
What is the maximum number of states that can be represented?

A) 5
B) 25
C) 31
D) 32

Answer

D

Explanation:

In Binary Encoding, each flip-flop stores 1 bit.

- Number of flip-flops = 5
- Number of possible states = $2^5 = 32$

3.An FSM has 8 states. Using Binary Encoding, how many flip-flops are required?

A) 2
B) 3
C) 4
D) 8

Answer

B) 3

Explanation:

$$\text{Flip-flops} = \lceil \log_2 N \rceil$$
$$\lceil \log_2 8 \rceil = \lceil 3 \rceil = 3$$



THANK YOU



DIGITAL LOGIC AND COMPUTER DESIGN

UE25CS152B

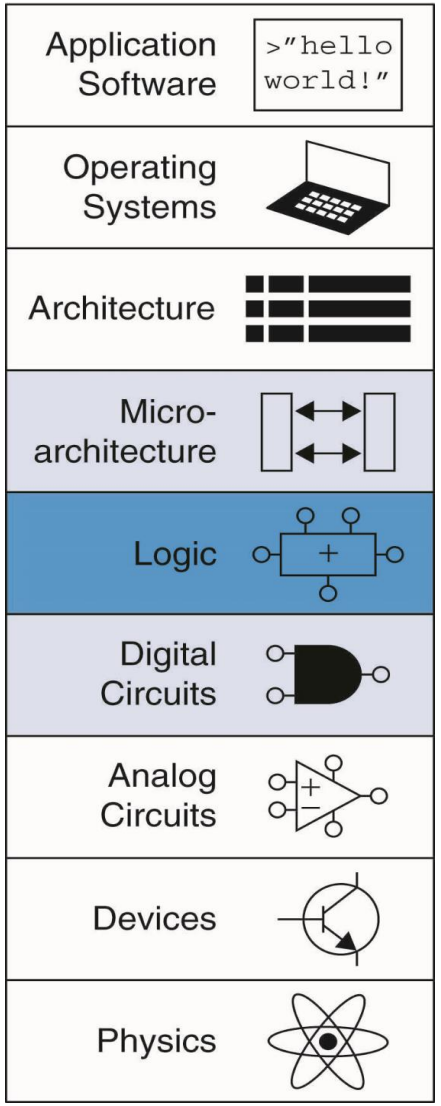
Introduction and Arithmetic Circuits

Team DLCD

Department of CSE and CSE(AI&ML)

Topics of Discussion

- Introduction
- Arithmetic Circuits
- Number Systems
- Sequential Building Blocks
- Memory Arrays
- Logic Arrays



DIGITAL BUILDING BLOCKS

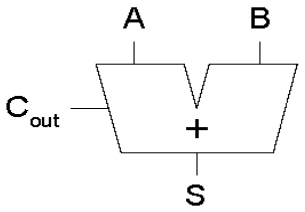
Introduction

- **Digital building blocks:**
  - Gates, multiplexers, decoders, registers, arithmetic circuits, counters, memory arrays, logic arrays
- **Building blocks demonstrate hierarchy, modularity, and regularity:**
  - Hierarchy of simpler components
  - Well-defined interfaces and functions
  - Regular structure easily extends to different sizes
- **Will use these building blocks in Chapter 7 to build microprocessor**

DIGITAL BUILDING BLOCKS

1-Bit Adders

Half Adder

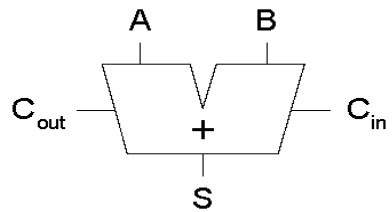


A	B	C_{out}	S
0	0		
0	1		
1	0		
1	1		

S =

C_{out} =

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

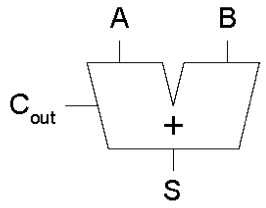
S =

C_{out} =

DIGITAL BUILDING BLOCKS

1-Bit Adders

Half Adder

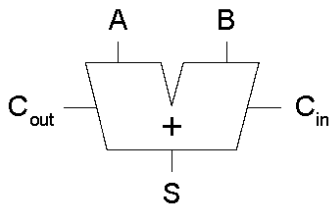


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

S =

C_{out} =

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

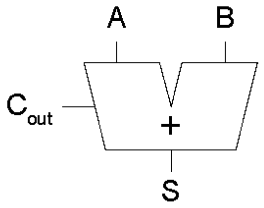
S =

C_{out} =

DIGITAL BUILDING BLOCKS

1-Bit Adders

Half Adder

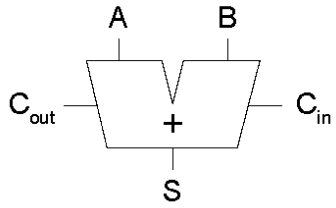


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S = A \oplus B$

$C_{out} = AB$

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S = A \oplus B \oplus C_{in}$

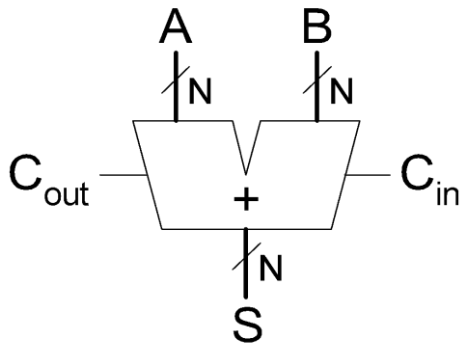
$C_{out} = AB + AC_{in} + BC_{in}$

DIGITAL BUILDING BLOCKS

Multibit Adders (CPAs)

- Types of carry propagate adders (CPAs):
  - Ripple-carry (slow)
  - Carry-lookahead (fast)
  - Prefix (faster)
- Carry-lookahead and prefix adders faster for large adders but require more hardware

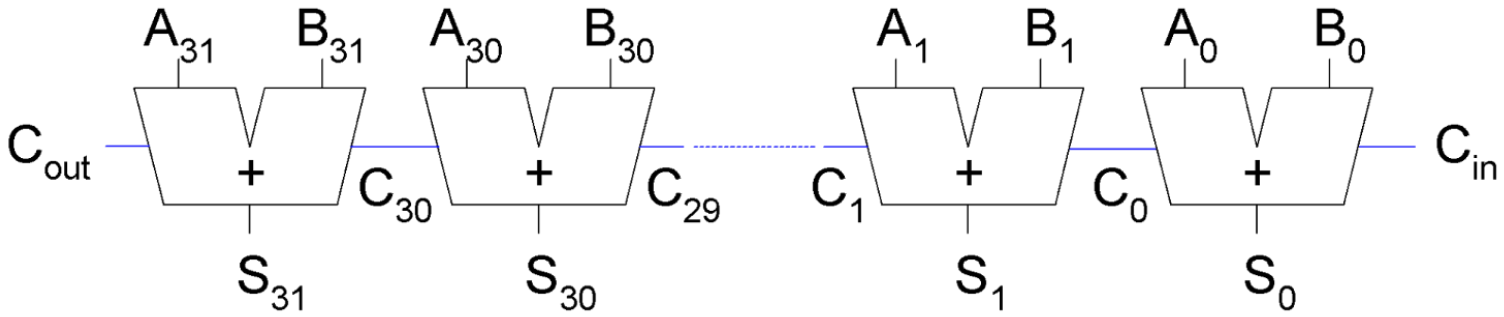
Symbol



DIGITAL BUILDING BLOCKS

Ripple-Carry Adder

- Chain 1-bit adders together
- Carry ripples through entire chain
- Disadvantage: **slow**



DIGITAL BUILDING BLOCKS

Ripple-Carry Adder Delay

$$t_{ripple} = Nt_{FA}$$

where t_{FA} is the delay of a 1-bit full adder

DIGITAL BUILDING BLOCKS

Carry-Lookahead Adder

- Compute carry out (C_{out}) for *k*-bit blocks using *generate* and *propagate* signals
- **Some definitions:**
  - Column *i* produces a carry out by either *generating* a carry out or *propagating* a carry in to the carry out
  - Generate (G_i) and propagate (P_i) signals for each column:
    - Column *i* will generate a carry out if A_i AND B_i are both 1.

$$G_i = A_i B_i$$

    - Column *i* will propagate a carry in to the carry out if A_i OR B_i is 1.

$$P_i = A_i + B_i$$

    - The carry out of column *i* (C_i) is:

$$C_i = A_i B_i + (A_i + B_i) C_{i-1} = G_i + P_i C_{i-1}$$

DIGITAL BUILDING BLOCKS

Carry-Lookahead Addition

- **Step 1:** Compute G_i and P_i for all columns
- **Step 2:** Compute G and P for k-bit blocks
- **Step 3:** C_{in} propagates through each k-bit propagate/generate block

DIGITAL BUILDING BLOCKS

Carry-Lookahead Adder

- **Example:** 4-bit blocks ($G_{3:0}$ and $P_{3:0}$) :

$$G_{3:0} = G_3 + P_3 (G_2 + P_2 (G_1 + P_1 G_0))$$

$$P_{3:0} = P_3 P_2 P_1 P_0$$

- **Generally,**

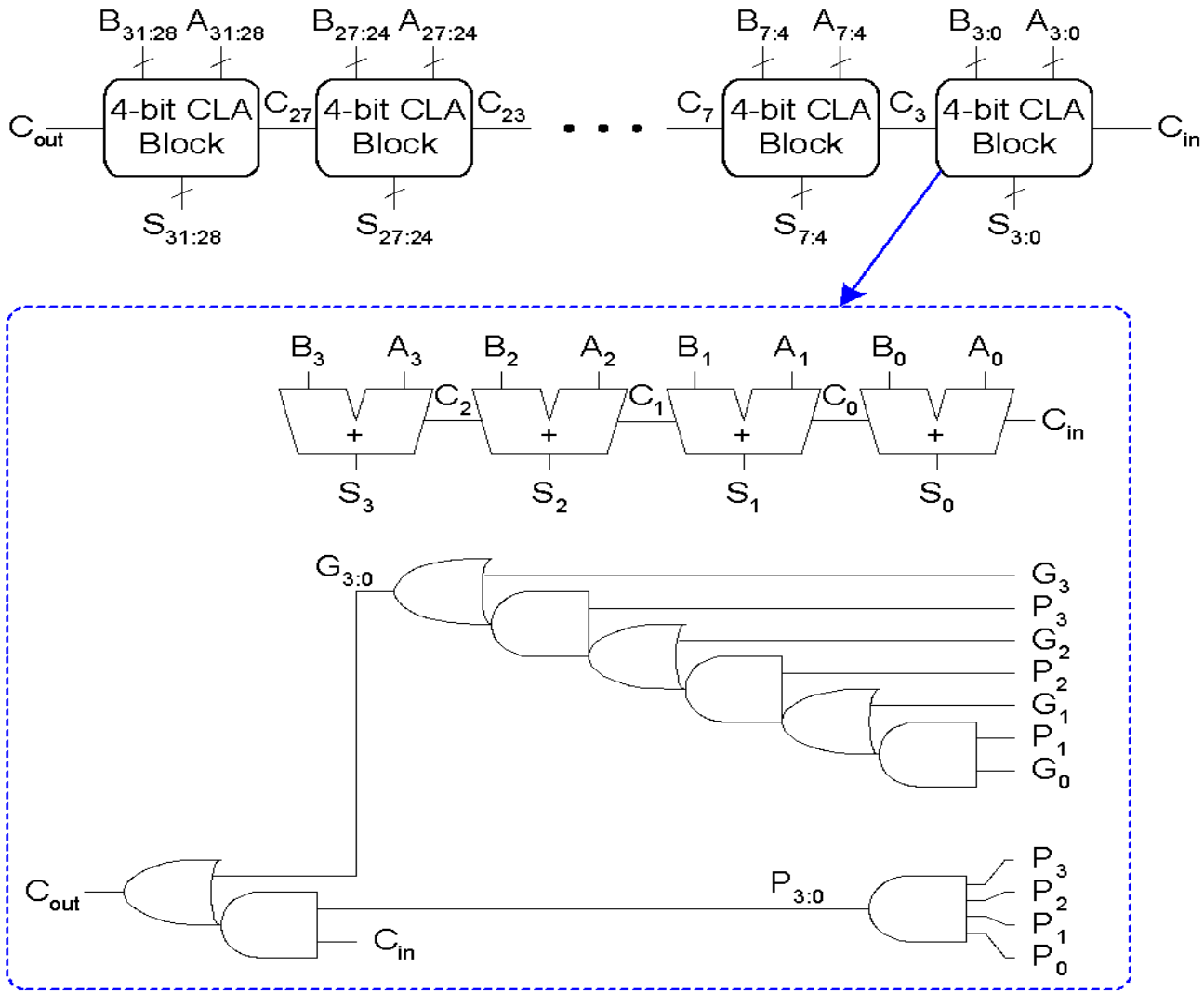
$$G_{i:j} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} G_j))$$

$$P_{i:j} = P_i P_{i-1} P_{i-2} P_j$$

$$C_i = G_{i:j} + P_{i:j} C_{j-1}$$

DIGITAL BUILDING BLOCKS

32-bit CLA with 4-bit Blocks



DIGITAL BUILDING BLOCKS

Carry-Lookahead Adder Delay

For N-bit CLA with k-bit blocks:

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$

- t_{pg} : delay to generate all P_i, G_i
- t_{pg_block} : delay to generate all $P_{i:j}$, $G_{i:j}$
- t_{AND_OR} : delay from C_{in} to C_{out} of final AND/OR gate in k-bit CLA block

An N-bit carry-lookahead adder is generally much faster than a ripple-carry adder for $N > 16$

DIGITAL BUILDING BLOCKS

Prefix Adder

- Computes carry in (C_{i-1}) for each column, then computes sum:

$$S_i = (A_i \oplus B_i) \oplus C_i$$

- Computes *G* and *P* for 1-, 2-, 4-, 8-bit blocks, etc. until all G_i (carry in) known
- $\log_2 N$ stages

DIGITAL BUILDING BLOCKS

Prefix Adder

- Carry in either *generated* in a column or *propagated* from a previous column.
- Column -1 holds C_{in}, so

  $$G_{-1} = C_{in},\ P_{-1} = 0$$

- Carry in to column i = carry out of column i-1:

  $$C_{i-1} = G_{i-1:-1}$$

  $G_{i-1:-1}$: generate signal spanning columns i-1 to -1
- Sum equation:

  $$S_i = (A_i \oplus B_i) \oplus G_{i-1:-1}$$

- **Goal:** Quickly compute $G_{0:-1}$, $G_{1:-1}$, $G_{2:-1}$, $G_{3:-1}$, $G_{4:-1}$, $G_{5:-1}$, … (called ***prefixes***)

DIGITAL BUILDING BLOCKS

Prefix Adder

- Generate and propagate signals for a block spanning bits *i*:*j*:

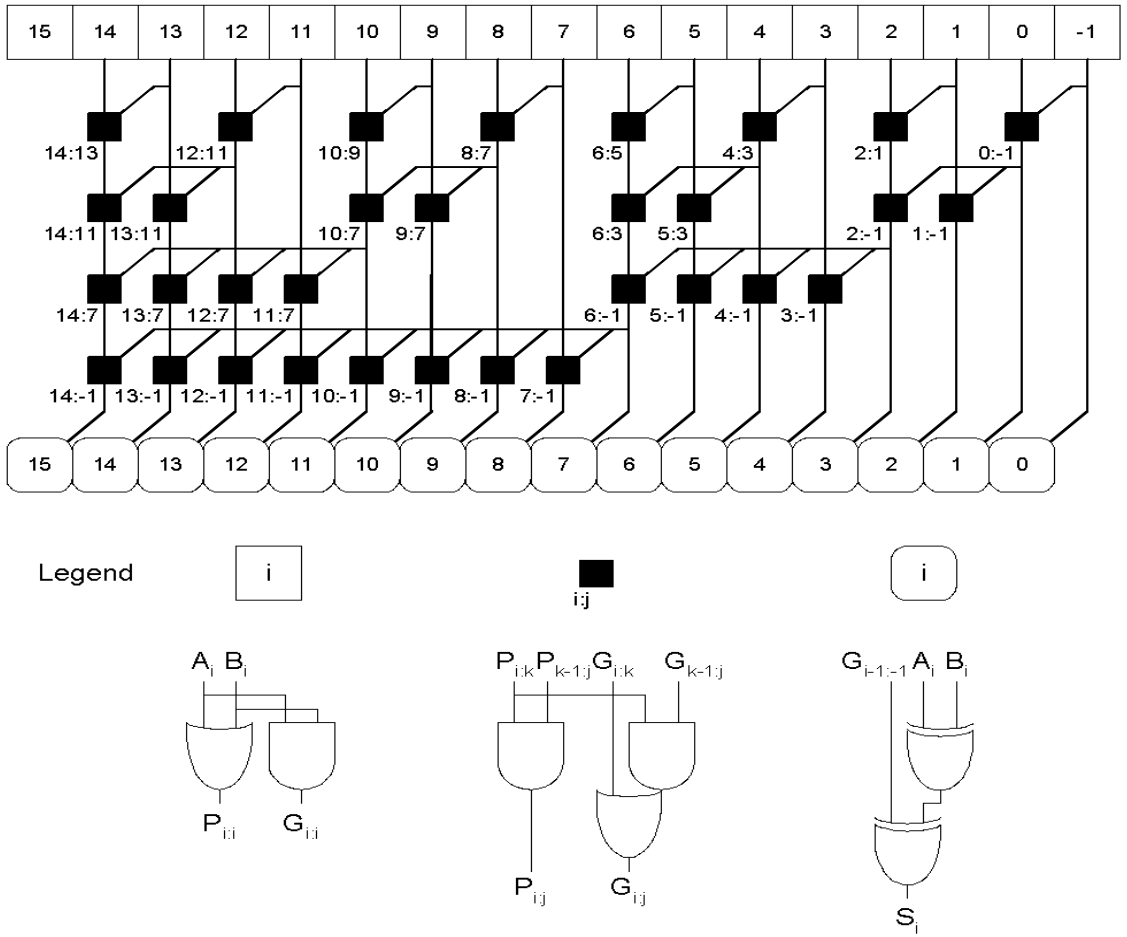
$$G_{i:j} = G_{i:k} + P_{i:k}\, G_{k-1:j}$$

$$P_{i:j} = P_{i:k} P_{k-1:j}$$

- In words:
  - **Generate:** block *i*:*j* will generate a carry if:
    - upper part (*i*:*k*) generates a carry or
    - upper part propagates a carry generated in lower part (*k*-1:*j*)
  - **Propagate:** block *i*:*j* will propagate a carry if *both* the upper and lower parts propagate the carry

DIGITAL BUILDING BLOCKS

Prefix Adder Schematic



DIGITAL BUILDING BLOCKS

Prefix Adder Delay

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$$

- t_{pg}: delay to produce P_i G_i (AND or OR gate)
- t_{pg_prefix}: delay of black prefix cell (AND-OR gate)

DIGITAL BUILDING BLOCKS

Adder Delay Comparisons

Compare delay of: 32-bit ripple-carry, carry-lookahead, and prefix adders

- CLA has 4-bit blocks
- 2-input gate delay = 100 ps; full adder delay = 300 ps

DIGITAL BUILDING BLOCKS

Adder Delay Comparisons

Compare delay of: 32-bit ripple-carry, carry-lookahead, and prefix adders

- CLA has 4-bit blocks
- 2-input gate delay = 100 ps; full adder delay = 300 ps

$$t_{ripple} = Nt_{FA} = 32(300\text{ ps})$$
$$= \mathbf{9.6\text{ ns}}$$

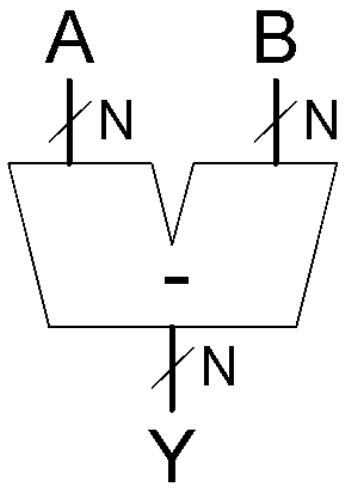
$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$
$$= [100 + 600 + (7)200 + 4(300)]\text{ ps}$$
$$= \mathbf{3.3\text{ ns}}$$

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$$
$$= [100 + \log_2 32(200) + 100]\text{ ps}$$
$$= \mathbf{1.2\text{ ns}}$$

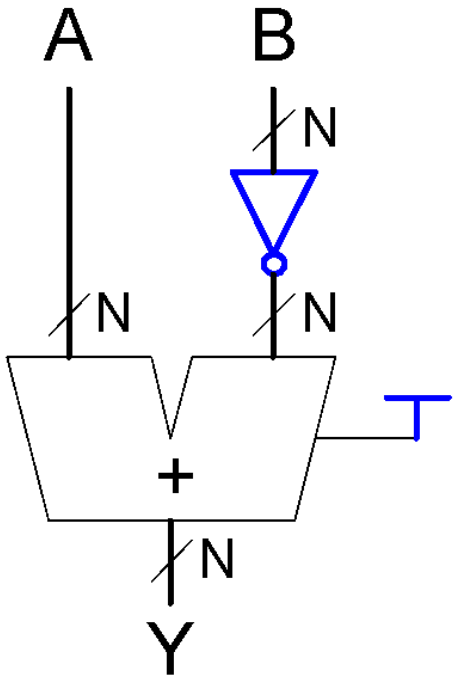
DIGITAL BUILDING BLOCKS

Subtracter

Symbol



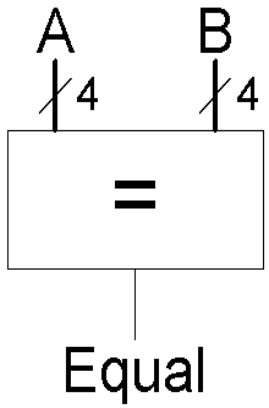
Implementation



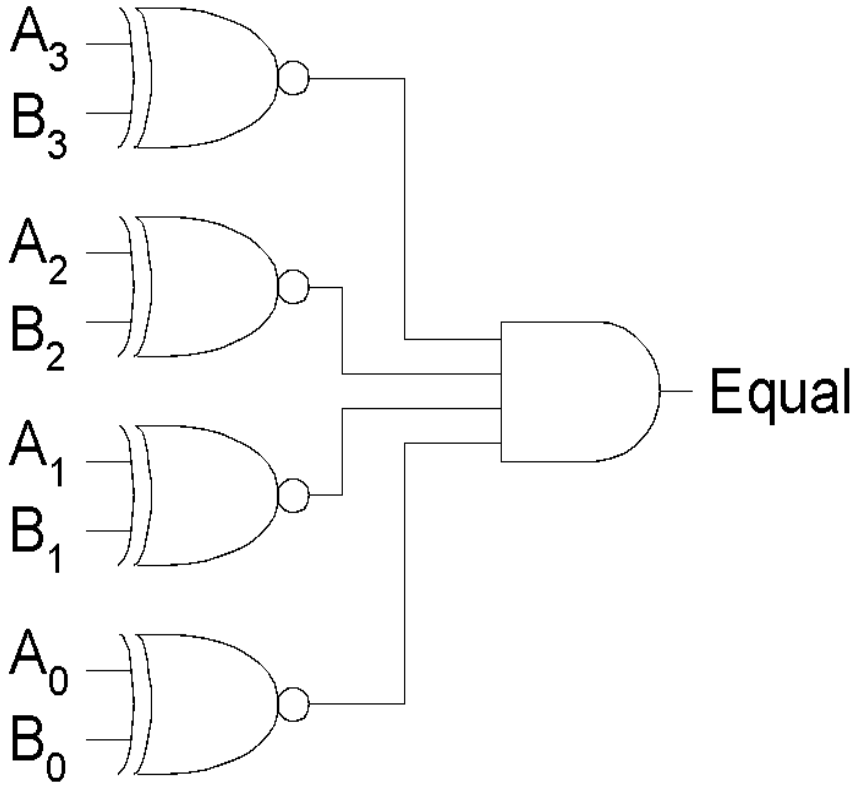
DIGITAL BUILDING BLOCKS

Comparator: Equality

Symbol

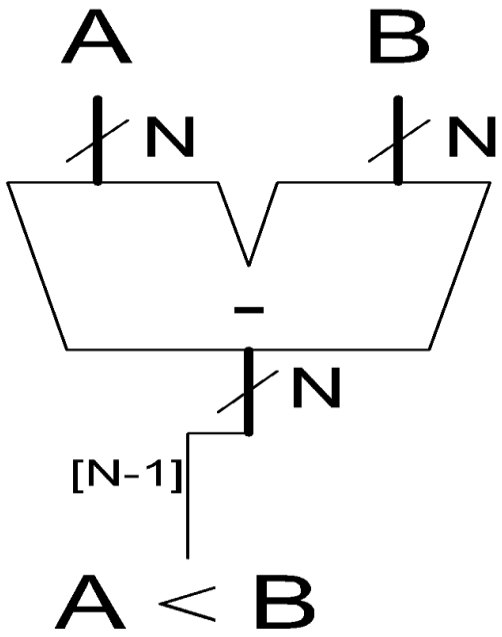


Implementation



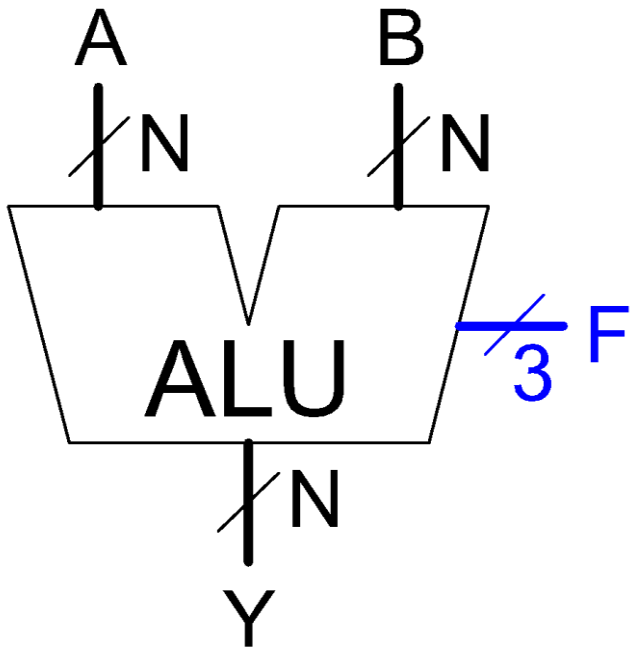
DIGITAL BUILDING BLOCKS

Comparator: Less Than



DIGITAL BUILDING BLOCKS

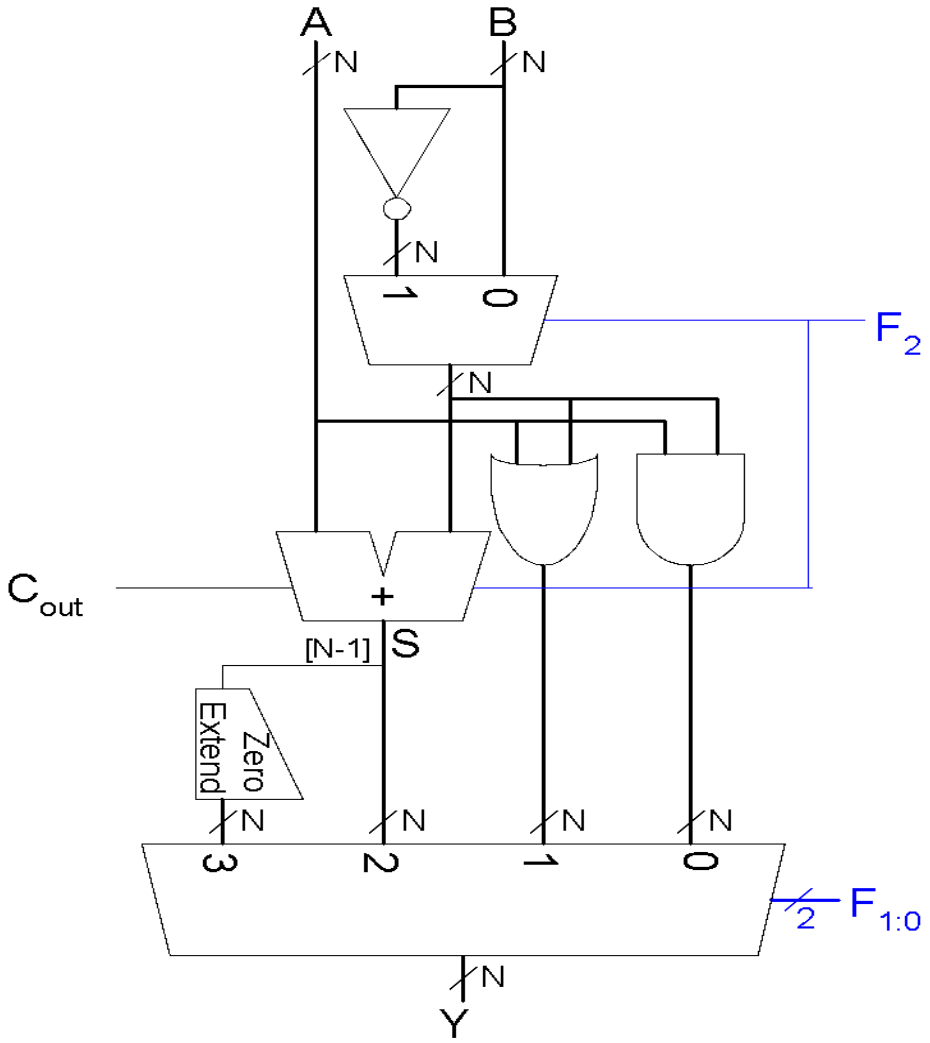
Arithmetic Logic Unit (ALU)



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

DIGITAL BUILDING BLOCKS

ALU Design



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

DIGITAL BUILDING BLOCKS

Adder Delay Comparisons

Compare delay of: 32-bit ripple-carry, carry-lookahead, and prefix adders

- CLA has 4-bit blocks
- 2-input gate delay = 100 ps; full adder delay = 300 ps

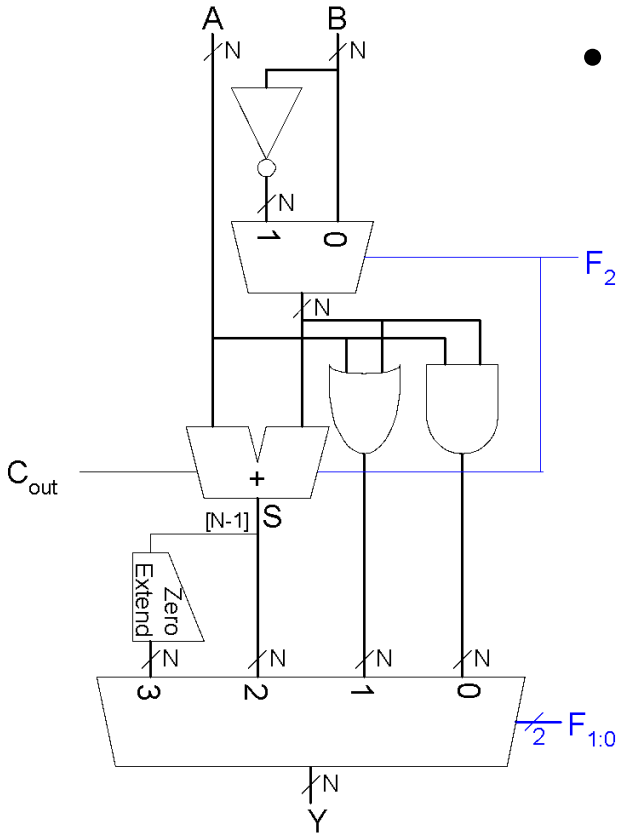
$$t_{ripple} = Nt_{FA} = 32(300\text{ ps})$$
$$= \mathbf{9.6\text{ ns}}$$

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + kt_{FA}$$
$$= [100 + 600 + (7)200 + 4(300)]\text{ ps}$$
$$= \mathbf{3.3\text{ ns}}$$

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$$
$$= [100 + \log_2 32(200) + 100]\text{ ps}$$
$$= \mathbf{1.2\text{ ns}}$$

DIGITAL BUILDING BLOCKS

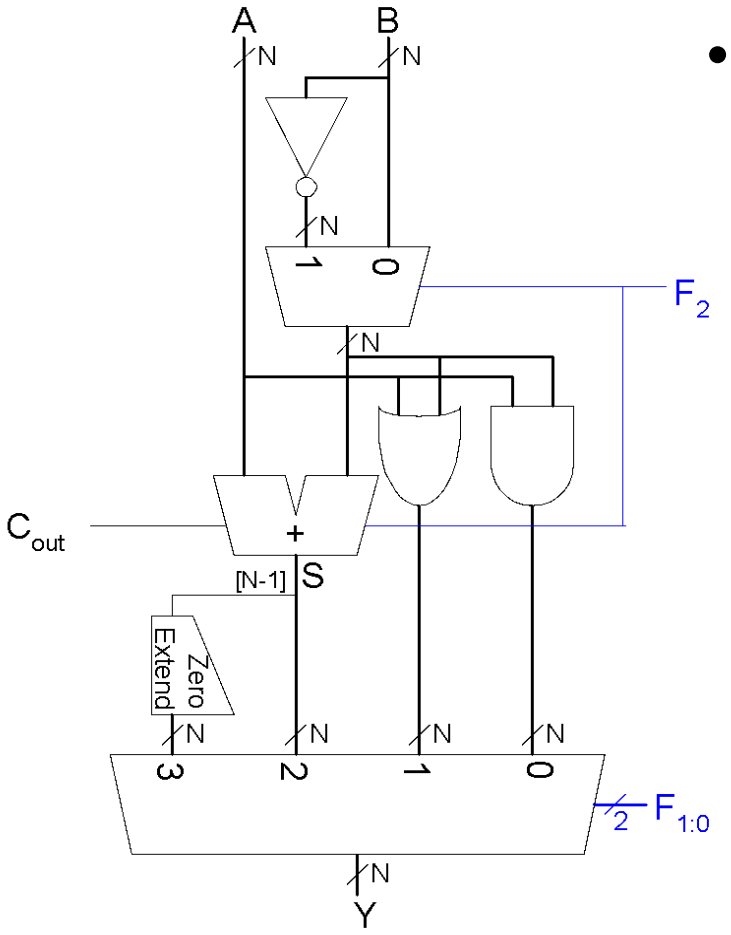
Set Less Than (SLT) Example



- Configure 32-bit ALU for SLT operation: *A* = 25 and *B* = 32

DIGITAL BUILDING BLOCKS

Set Less Than (SLT) Example



- Configure 32-bit ALU for SLT operation: A = 25 and B = 32
  - ***A* < *B***, so Y should be 32-bit representation of 1 (0x00000001)
  - $F_{2:0}$ **= 111**
    - F_2 **= 1** (adder acts as subtracter), so 25 - 32 = -7
    - -7 has 1 in the most significant bit (S_{31} = 1)
    - $F_{1:0}$ **= 11** multiplexer selects Y = S_{31} (zero extended) = 0x00000001.

DIGITAL BUILDING BLOCKS

Shifters

- **Logical shifter:** shifts value to left or right and fills empty spaces with 0's
  - Ex: 11001 >> 2 =
  - Ex: 11001 << 2 =

- **Arithmetic shifter:** same as logical shifter, but on right shift, fills empty spaces with the old most significant bit (msb).
  - Ex: 11001 >>> 2 =
  - Ex: 11001 <<< 2 =

- **Rotator:** rotates bits in a circle, such that bits shifted off one end are shifted into the other end
  - Ex: 11001 ROR 2 =
  - Ex: 11001 ROL 2 =

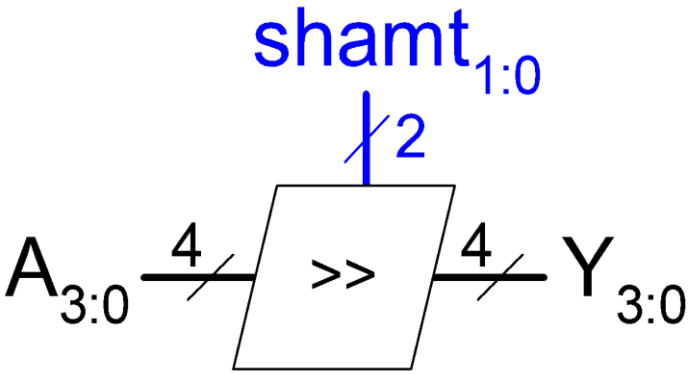
DIGITAL BUILDING BLOCKS

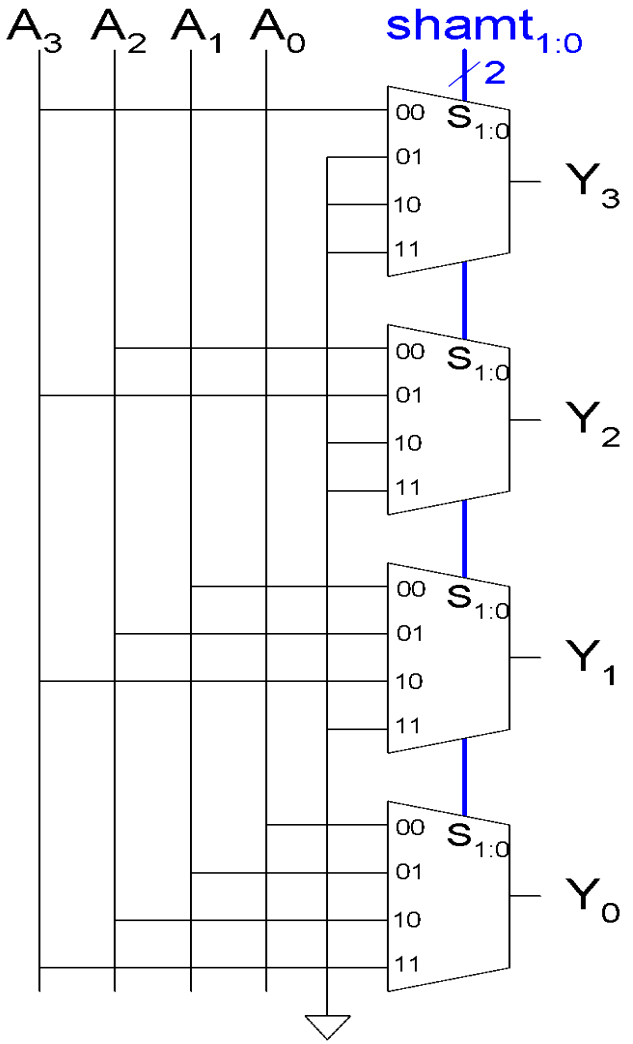
Shifters

- **Logical shifter:**
  - Ex: 11001 >> 2 = 00110
  - Ex: 11001 << 2 = 00100
- **Arithmetic shifter:**
  - Ex: 11001 >>> 2 = 11110
  - Ex: 11001 <<< 2 = 00100
- **Rotator:**
  - Ex: 11001 ROR 2 = 01110
  - Ex: 11001 ROL 2 = 00111

DIGITAL BUILDING BLOCKS

Shifter Design





DIGITAL BUILDING BLOCKS

Shifters as Multipliers,Dividers

- $A << N = A \times 2^N$
  - **Example:** 00001 << 2 = 00100 $(1 \times 2^2 = 4)$
  - **Example:** 11101 << 2 = 10100 $(-3 \times 2^2 = -12)$
- $A >>> N = A \div 2^N$
  - **Example:** 01000 >>> 2 = 00010 $(8 \div 2^2 = 2)$
  - **Example:** 10000 >>> 2 = 11100 $(-16 \div 2^2 = -4)$

DIGITAL BUILDING BLOCKS

Multipliers

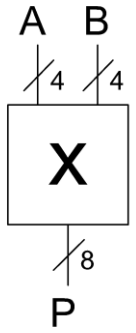
- **Partial products** formed by multiplying a single digit of the multiplier with multiplicand
- **Shifted** partial products **summed** to form result

Decimal		Binary
230	multiplicand	0101
x 42	multiplier	x 0111
460	partial products	0101
+ 920		0101
9660		0101
		+ 0000
	result	0100011

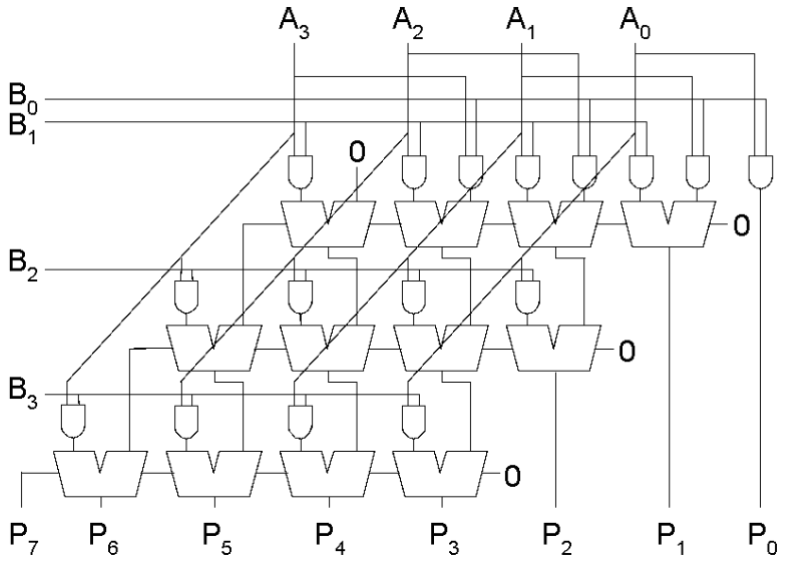
230 x 42 = 9660 5 x 7 = 35

DIGITAL BUILDING BLOCKS

4 x 4 Multiplier

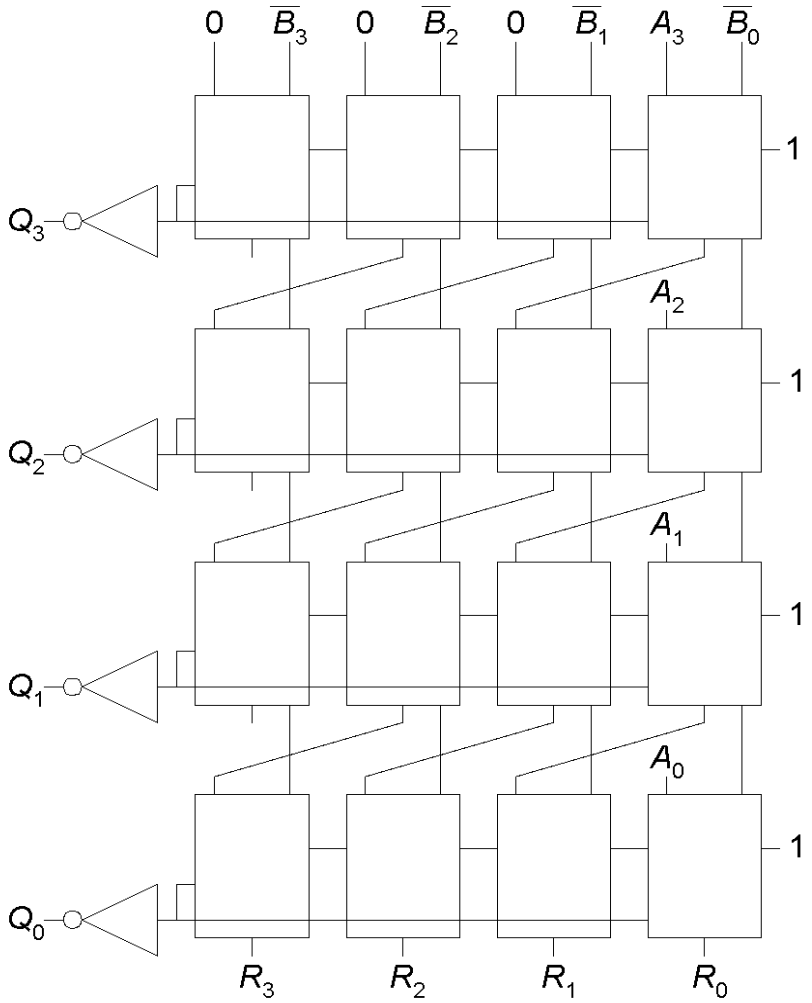


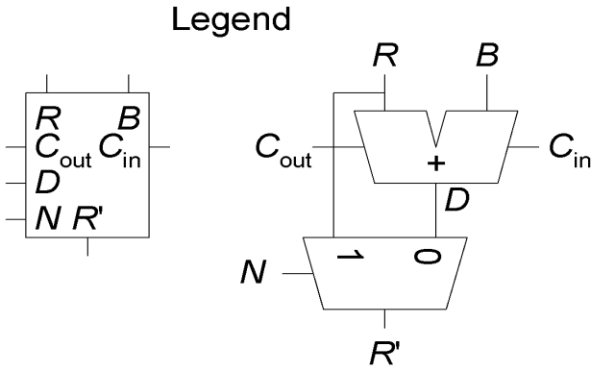
$$
\begin{array}{cccccccc}
 & & A_3 & A_2 & A_1 & A_0 & & \\
\times & & B_3 & B_2 & B_1 & B_0 & & \\
\hline
 & & A_3B_0 & A_2B_0 & A_1B_0 & A_0B_0 & & \\
 & A_3B_1 & A_2B_1 & A_1B_1 & A_0B_1 & & & \\
 & A_3B_2 & A_2B_2 & A_1B_2 & A_0B_2 & & & \\
+ & A_3B_3 & A_2B_3 & A_1B_3 & A_0B_3 & & & \\
\hline
P_7 & P_6 & P_5 & P_4 & P_3 & P_2 & P_1 & P_0
\end{array}
$$



DIGITAL BUILDING BLOCKS

4 x 4 Divider





A/B = Q + R/B

Algorithm:

R’ = 0

for *i* = N-1 to 0

  R = {R’ << 1. A_i}

  D = R - B

  if D < 0, Q_i=0, R’=R

  else Q_i=1, R’=D

R’=R

MCQs

Q1. What are the different types of Carry propagator adders?

a) Ripple carry adders

b) Carry lookahead adders

c) Prefix adders

d) Skip ahead adders

MCQs

Q1. What are the different types of Carry propagator adders?

a) **Ripple carry adders**

b) **Carry lookahead adders**

c) **Prefix adders**

d) Skip ahead adders

MCQs

Q2. A comparator is used to:

a. Add two numbers

b. Multiply two numbers

c. Determine relative magnitude of two numbers

d. Shift a number

MCQs

Q2. A comparator is used to:

a. Add two numbers

b. Multiply two numbers

c. Determine relative magnitude of two numbers

d. Shift a number

MCQs

Q3. Which circuit performs arithmetic and logical operations?

a. Decoder

b. Multiplexer

c. ALU

d. Counter

MCQs

Q3. Which circuit performs arithmetic and logical operations?

a. Decoder

b. Multiplexer

c. **ALU**

d. Counter



THANK YOU

Team DLCD

Department of CSE and CSE(AI&ML)



DIGITAL LOGIC AND COMPUTER DESIGN

DIGITAL BUILDING BLOCKS

Number Systems

Team DLCD

Department of CSE and AIML

DIGITAL BUILDING BLOCKS

Number Systems

Team DLCD

DIGITAL BUILDING BLOCKS

Number Systems

Numbers we can represent using binary representations

-Positive numbers
- Unsigned binary

-Negative numbers
- Two's complement
- Sign/magnitude numbers

What about fractions?

Two common notations:

- **Fixed-point:** binary point fixed
- **Floating-point:** binary point floats to the right of the most significant 1

DIGITAL BUILDING BLOCKS

Fixed-Point Numbers

6.75 using 4 integer bits and 4 fraction bits:

$$01101100$$

$$0110.1100$$

$$2^2 + 2^1 + 2^{-1} + 2^{-2} = 6.75$$

-Binary point is implied

-The number of integer and fraction bits must be agreed upon beforehand

Represent 7.5_{10} using 4 integer bits and 4 fraction bits.

Represent 7.5_{10} using 4 integer bits and 4 fraction bits.

Answer: 01111000

DIGITAL BUILDING BLOCKS

Signed Fixed-Point Numbers

Representations:

- Sign/magnitude
- Two's complement

Example: Represent -7.5_{10} using 4 integer and 4 fraction bits

- **Sign/magnitude:**

- **Two's complement:**

DIGITAL BUILDING BLOCKS

Signed Fixed-Point Numbers

Representations:

- Sign/magnitude
- Two's complement

Example: Represent -7.5_{10} using 4 integer and 4 fraction bits

- **Sign/magnitude:**

  11111000

- **Two's complement:**

  1. +7.5: 01111000
  2. Invert bits: 10000111
  3. Add 1 to LSB: + 1

  10001000

DIGITAL BUILDING BLOCKS

Floating-Point Numbers

- Binary point floats to the right of the most significant 1
- Similar to decimal scientific notation

Example: write 273_{10} in scientific notation:

$$273 = 2.73 \times 10^2$$

In general, a number is written in scientific notation as:

$$\pm M \times B^E$$

- **M** = mantissa
- **B** = base
- **E** = exponent
- In the example, M = 2.73, B = 10, and E = 2

DIGITAL BUILDING BLOCKS

Floating-Point Numbers



We show three versions –final version is called the **IEEE 754 floating-point standard**

Example: represent the value 228_{10} using a 32-bit floating point representation

DIGITAL BUILDING BLOCKS

Floating-Point Representation 1

Example: represent the value 228_{10} using a 32-bit floating point representation

1. Convert decimal to binary (**don't reverse steps 1 & 2!**): $228_{10} = 11100100_2$

2. Write the number in "binary scientific notation": $11100100_2 = 1.11001_2 \times 2^7$

3. Fill in each field of the 32-bit floating point number:

   -The sign bit is positive (0)

   -The 8 exponent bits represent the value 7

   -The remaining 23 bits are the mantissa

1 bit	8 bits	23 bits
0	00000111	11 1001 0000 0000 0000 0000
Sign	**Exponent**	**Mantissa**

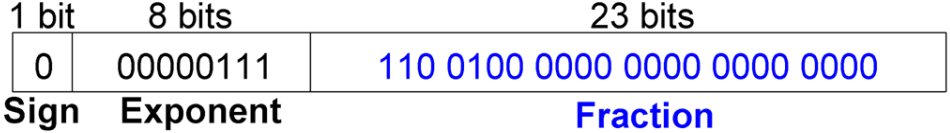
DIGITAL BUILDING BLOCKS

Floating-Point Representation 2

- First bit of the mantissa is always 1:

$$-228_{10} = 11100100_2 = 1.11001 \times 2^7$$

- So, no need to store it: *implicit leading 1*
- Store just fraction bits in 23-bit field



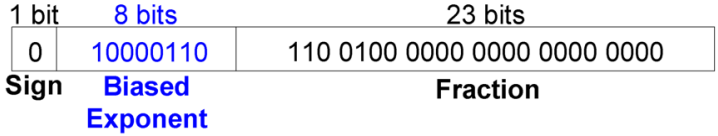
DIGITAL BUILDING BLOCKS

Floating-Point Representation 3

Biased exponent: bias = 127 (01111111_2)

- Biased exponent = bias + exponent
- Exponent of 7 is stored as: 127 + 7 = 134 = $0x10000110_2$

The **IEEE 754 32-bit floating-point representation** of 228_{10}



In hexadecimal: 0x43640000

DIGITAL BUILDING BLOCKS

Floating-Point Example

Write -58.25_{10} in floating point (IEEE 754)

DIGITAL BUILDING BLOCKS

Floating-Point Example

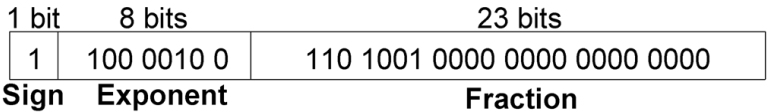
Write -58.25_{10} in floating point (IEEE 754)

1. Convert decimal to binary: $58.25_{10} = 111010.01_2$

2. Write in binary scientific notation: 1.1101001×2^5

3. Fill in fields:

   Sign bit: 1 (negative)
   8 exponent bits: $(127 + 5) = 132 = 10000100_2$
   23 fraction bits: 110 1001 0000 0000 0000 0000



In hexadecimal: **0xC2690000**

DIGITAL BUILDING BLOCKS

Floating-Point: Special Cases

PES UNIVERSITY

Number	Sign	Exponent	Fraction
0	X	00000000	00000000000000000000000
∞	0	11111111	00000000000000000000000
- ∞	1	11111111	00000000000000000000000
NaN	X	11111111	non-zero

DIGITAL BUILDING BLOCKS

Floating-Point Precision

Single-Precision:

- 32-bit
- 1 sign bit, 8 exponent bits, 23 fraction bits
- bias = 127

Double-Precision:

- 64-bit
- 1 sign bit, 11 exponent bits, 52 fraction bits
- bias = 1023

DIGITAL BUILDING BLOCKS

Floating-Point: Rounding

- **Overflow:** number too large to be represented
- **Underflow:** number too small to be represented

- **Rounding modes:**
  - Down
  - Up
  - Toward zero
  - To nearest

DIGITAL BUILDING BLOCKS

Floating-Point: Rounding

Example: Round 1.100101 (1.578125) to only 3 fraction bits

Down: 1.100

Up: 1.101

Toward zero: 1.100

To nearest: 1.101 (1.625 is closer to 1.578125 than 1.5 is)

DIGITAL BUILDING BLOCKS

Floating-Point Addition

PES UNIVERSITY
CELEBRATING 50 YEARS

1. Extract exponent and fraction bits
2. Prepend leading 1 to form mantissa
3. Compare exponents
4. Shift smaller mantissa if necessary
5. Add mantissas
6. Normalize mantissa and adjust exponent if necessary
7. Round result
8. Assemble exponent and fraction back into floating-point format

DIGITAL BUILDING BLOCKS

Floating-Point Addition Example

Add the following floating-point numbers:

0x3FC00000

0x40500000

DIGITAL BUILDING BLOCKS

Floating-Point Addition Example

1. Extract exponent and fraction bits

1 bit	8 bits	23 bits
0	01111111	100 0000 0000 0000 0000 0000
Sign	**Exponent**	**Fraction**

1 bit	8 bits	23 bits
0	10000000	101 0000 0000 0000 0000 0000
Sign	**Exponent**	**Fraction**

For first number (N1): S = 0, E = 127, F = .1

For second number (N2): S = 0, E = 128, F = .101

2. Prepend leading 1 to form mantissa

N1: 1.1

N2: 1.101

Floating-Point Addition Example

3. Compare exponents

127 – 128 = -1, so shift N1 right by 1 bit

4. Shift smaller mantissa if necessary

shift N1's mantissa: 1.1 >> 1 = 0.11 $(\times 2^1)$

5. Add mantissas

$$\begin{array}{r} 0.11 \times 2^1 \\ +\ 1.101 \times 2^1 \\ \hline 10.011 \times 2^1 \end{array}$$

6. Normalize mantissa and adjust exponent if necessary

$10.011 \times 2^1 = 1.0011 \times 2^2$

7. Round result

No need (fits in 23 bits)

8. Assemble exponent and fraction back into floating-point format

$S = 0$, $E = 2 + 127 = 129 = 10000001_2$, $F = 001100..$

1 bit	8 bits	23 bits
0	10000001	001 1000 0000 0000 0000 0000
Sign	**Exponent**	**Fraction**

In hexadecimal: **0x40980000**

DIGITAL BUILDING BLOCKS

MCQs

1. Which of the following is NOT a representation for signed fixed-point numbers?

A. Two's Complement
B. Sign/Magnitude
C. Excess-127
D. Both A and B

2. In IEEE 754 single-precision floating-point format, how many bits are allocated for the exponent?

A. 1
B. 8
C. 11
D. 23

DIGITAL BUILDING BLOCKS

MCQs

3. Which IEEE 754 representation corresponds to NaN (Not a Number)?

A. Exponent = 00000000, Fraction = 000...0
B. Exponent = 11111111, Fraction = 000...0
C. Exponent = 11111111, Fraction ≠ 0
D. Exponent = 00000000, Fraction ≠ 0

4. During floating-point addition, what is the first operation performed after extracting the exponent and fraction bits?

A. Normalize the mantissa
B. Round the result
C. Prepend the implicit leading 1 to form the mantissa
D. Compare the exponents

DIGITAL BUILDING BLOCKS

MCQs

1. **Answer: C. Excess-127**
   Explanation: The PPT lists **Sign/Magnitude** and **Two's Complement** as signed fixed-point representations.

2. **Answer: B. 8**
   Explanation: Single-precision uses **1 sign bit, 8 exponent bits, and 23 fraction (mantissa) bits**.

3. **Answer: C. Exponent = 11111111, Fraction ≠ 0**
   Explanation: NaN is represented by an all-ones exponent with a non-zero fraction field.

4. **Answer: C. Prepend the implicit leading 1 to form the mantissa**
   Explanation: After extracting the fields, the hidden leading **1** is restored before comparing exponents and performing the addition.



THANK YOU

Team DLCD
Department of Computer Science & AIML